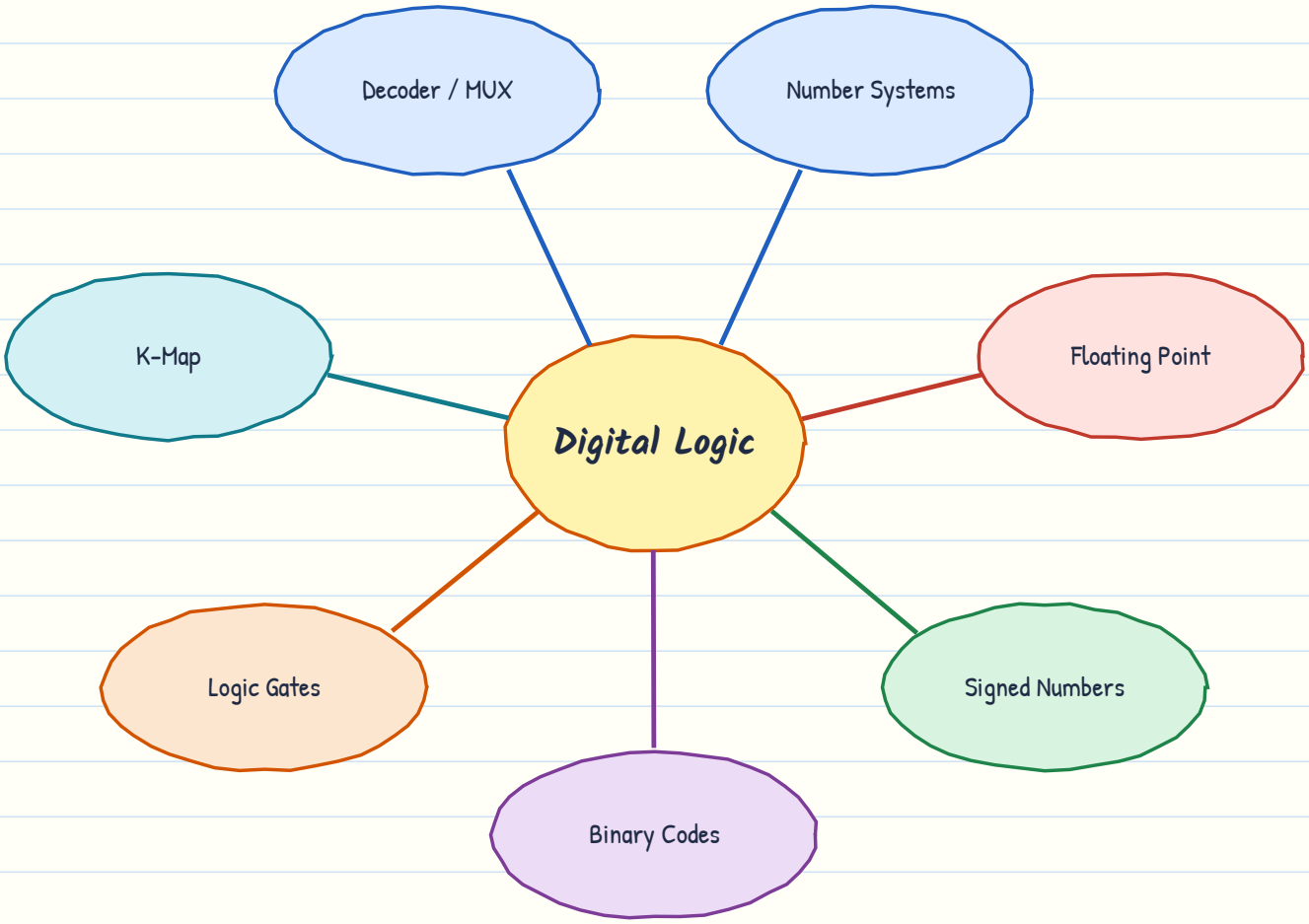
UNIT 1

Data Representation & Digital Logic Circuits-1

Number systems | Complements | Codes | Gates | K-map | Adders | Decoders | MUX



Mind map: everything you will learn in this unit

How to use these notes

Tip: first understand each diagram, then re-draw gates and K-maps on paper yourself - this is how digital logic is tested in exams.

1. Number Systems & Conversions

Computers store everything as **bits** (0 or 1). A number system is defined by its **base (radix)** - the count of digits it uses.

System	Base	Digits	Example
Binary	2	0, 1	1011
Octal	8	0 - 7	17
Decimal	10	0 - 9	45
Hexadecimal	16	0-9, A-F	2F

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Decimal -> Binary (repeated division by 2)

```
2 | 25    -> remainder 1
2 | 12    -> remainder 0
2 |  6    -> remainder 0
2 |  3    -> remainder 1
2 |  1    -> remainder 1

= 11001
```

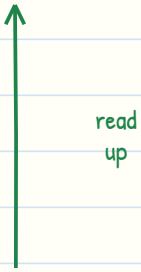


Fig 1: (25)10 = (11001)2 - read remainders bottom to top

- **Fraction:** multiply by 2 repeatedly, take the integer parts top to bottom. 0.625 -> 1.25(1), 0.5(0), 1.0(1) = **0.101**
- **Binary -> Decimal:** multiply each bit by its weight (powers of 2): 1101 = 8+4+0+1 = **13**

Octal & Hexadecimal shortcuts

1 0 1 1 1 0 1 0 1 1

Octal: 001|011|101|011

= 1 3 5 3 = (1353)8

Hex: 0010|1110|1011

= 2 E B = (2EB)16

Fig 2: group bits - 3 for octal, 4 for hex

Rule

Group from the RIGHT (the binary point for fractions: left of point groups from the point to the left, right side from the point to the right). Pad with 0s.

2. Fixed & Floating Point Representation

Fixed point: the binary point is at a fixed position (usually at the right end - integers, or after the sign bit - fractions). Simple, but small range.

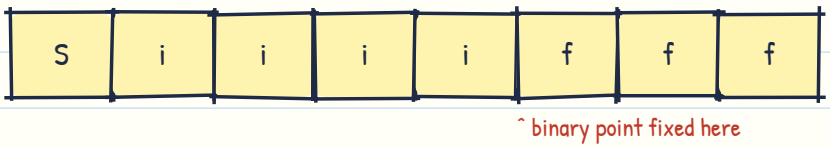


Fig 3: 8-bit fixed point (sign | 4 integer | 3 fraction)

Floating point: like scientific notation, number = $(-1)^S \times 1.M \times 2^{(E-bias)}$. Stores a wide range of values.

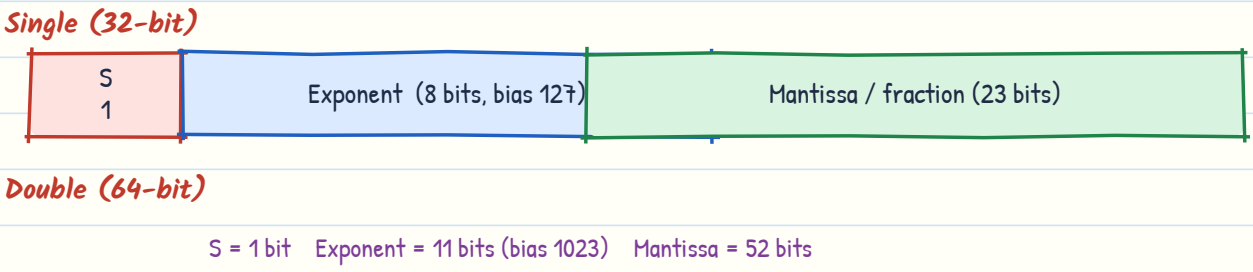


Fig 4: IEEE 754 formats

Example: store 12.5 in IEEE 754 single precision

Step	Work
1. Binary	12.5 = 1100.1
2. Normalize	1.1001 x 2^3 (move point 3 places)
3. Sign	positive -> S = 0
4. Exponent	3 + 127 = 130 = 10000010

5. Mantissa	1001 followed by 0s (hidden leading 1 dropped)
Result	0 10000010 10010000000000000000000 = 0x41480000

Remember

Normalized form always has a hidden leading 1, so only the digits AFTER the point (mantissa) are stored. Exponent is stored with a BIAS so negative exponents need no sign.

3. Signed Binary Numbers & Complements

To store negative numbers, the ***MSB is the sign bit*** (0 = positive, 1 = negative). Three methods:

Value	Sign-magnitude	1's complement	2's complement
+5	0101	0101	0101
-5	1101	1010	1011
+0	0000	0000	0000
-0	1000	1111	(none)
Range (4-bit)	-7 to +7	-7 to +7	-8 to +7

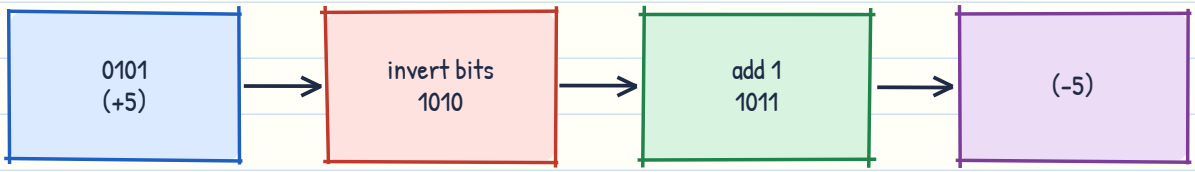


Fig 5: How to get 2's complement of 0101 (+5)

- ***1's complement*** = flip every bit. ***2's complement*** = 1's complement + 1.
- 2's complement is used in computers: only one zero and subtraction becomes addition.

Subtraction using 2's complement

```
  0111   (7)
+ 1011   (-5 in 2's)
-------------------
1 0010   -> discard carry

answer = 0010 = +2
```

Fig 6: 7 - 5 = 7 + (-5)

Carry vs overflow

If a carry goes out of the MSB it is DISCARDED. Overflow occurs when two numbers of the same sign give a result of the opposite sign.

r's and (r-1)'s complements

Base	(r-1)'s complement	r's complement
Binary (r=2)	1's complement	2's complement
Decimal (r=10)	9's complement	10's complement
Method	subtract each digit from r-1	(r-1)'s + 1

4. Binary Codes

Dec	BCD (8421)	Excess-3	Gray
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

- **BCD:** each decimal digit coded by 4 bits (1010-1111 unused). 25 = 0010 0101.
- **Excess-3:** BCD + 3; self-complementing code.
- **Gray code:** only ONE bit changes between consecutive values - used in encoders to avoid errors.
- **ASCII:** 7-bit code for characters (A = 65 = 1000001, a = 97, 0 = 48).

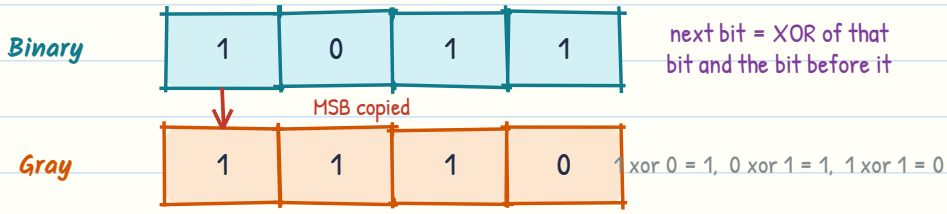


Fig 7: Binary -> Gray conversion (1011 -> 1110)

Error detection - parity bit

An extra bit is added so total 1s are **even (even parity)** or **odd**. Data 1011 has three 1s -> even-parity bit = **1** -> 11011. Detects any **single-bit** error.

5. Digital Logic Circuits - 1

Basic logic functions & gates

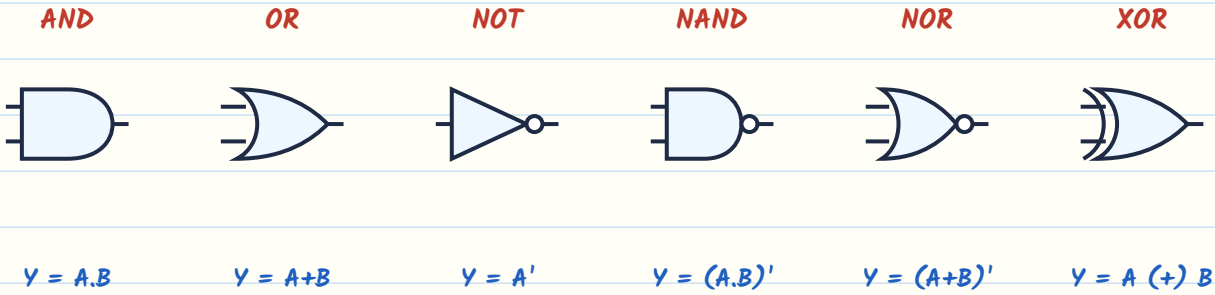


Fig 8: Logic gates and truth tables

A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT A
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

- **AND:** output 1 only if ALL inputs are 1.
- **OR:** output 1 if ANY input is 1.
- **NOT:** inverts the input.
- **XOR:** 1 when inputs are DIFFERENT; **XNOR:** 1 when inputs are SAME.

Universal gates (NAND and NOR)

NAND and **NOR** are called ***universal gates*** because any logic function can be built using only one of them.

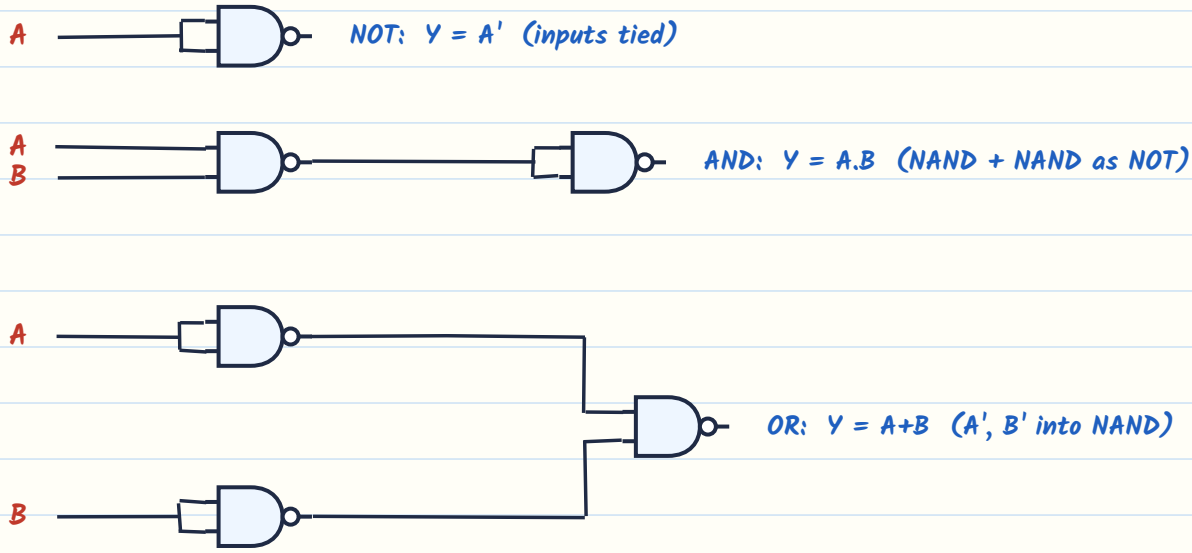


Fig 9: NOT, AND, OR built from NAND gates only

Required gate	Using NAND	Using NOR
NOT	NAND with inputs tied	NOR with inputs tied
AND	NAND -> NAND(as NOT)	invert both inputs, then NOR
OR	invert both inputs, then NAND	NOR -> NOR(as NOT)

Boolean algebra laws

Law	AND form	OR form
Identity	A.1 = A	A+0 = A
Null	A.0 = 0	A+1 = 1
Idempotent	A.A = A	A+A = A
Complement	$A.A' = 0$	$A+A' = 1$
Commutative	A.B = B.A	A+B = B+A
Distributive	A(B+C)=AB+AC	A+BC=(A+B)(A+C)
Absorption	A(A+B) = A	A+AB = A
De Morgan	$(AB)' = A'+B'$	$(A+B)' = A'B'$

Most asked!

De Morgan: "break the line, change the sign". NAND = OR of inverted inputs; NOR = AND of inverted inputs.

Minimization of logic expressions

Goal: fewer gates, cheaper and faster circuit. Forms: **SOP** (sum of products, minterms, e.g. AB + A'C) and **POS** (product of sums, maxterms).

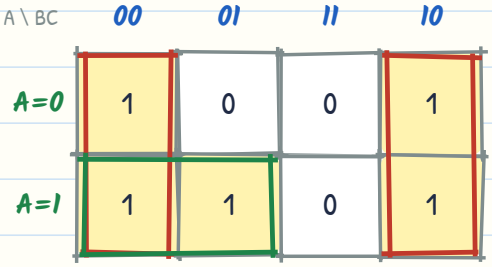
Algebraic simplification

```
Example:  F = AB + AB' + A'B
        = A(B + B') + A'B      [distributive]
        = A.1 + A'B            [complement]
        = A + A'B = A + B      [absorption variant]
```

Karnaugh map (K-map) simplification

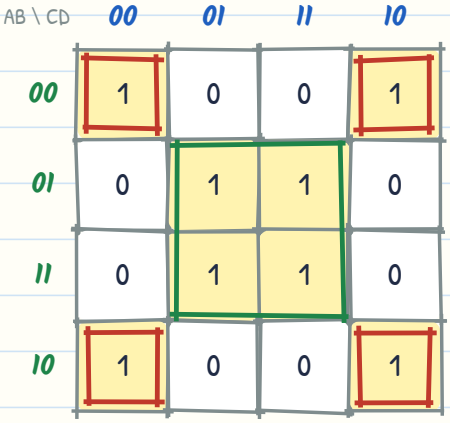
A K-map is a grid of minterms arranged in **Gray-code order** so that neighbouring cells differ by one variable. Group adjacent 1s in **powers of 2** (1,2,4,8,16).

- Groups must be rectangles (wrap-around allowed at edges); make groups as **big** as possible.
- Every 1 must be covered; overlap is allowed.
- A group eliminates the variables that change inside it.



quad (red) = C'
pair (green) = AB'

Fig 10: 3-variable K-map F = SUM m(0,2,4,5,6) -> F = C' + AB'



corners (red) = B'D'

centre (green) = BD

Fig 11: 4-variable K-map F = SUM m(0,2,5,7,8,10,13,15) -> F = B'D' + BD

Bonus tip

Don't-care conditions (X) may be used as 1 or 0 - whichever gives bigger groups.

6. Combinational Circuits

Output depends ***only on present inputs*** (no memory). Examples: adders, decoders, multiplexers, encoders.

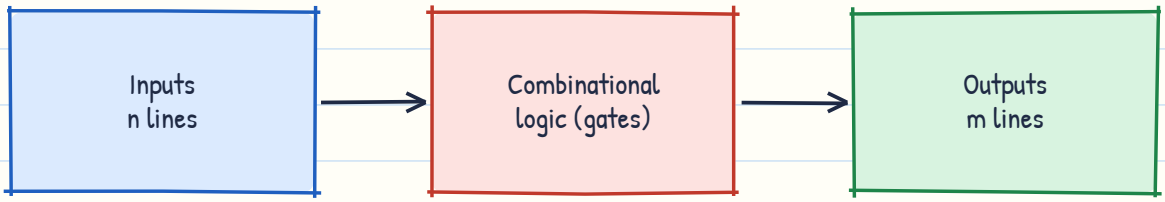


Fig 12: Combinational circuit block

Half adder & full adder

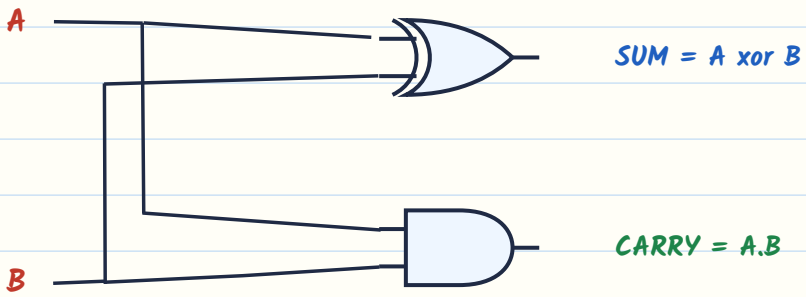


Fig 13: Half adder (adds two bits)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

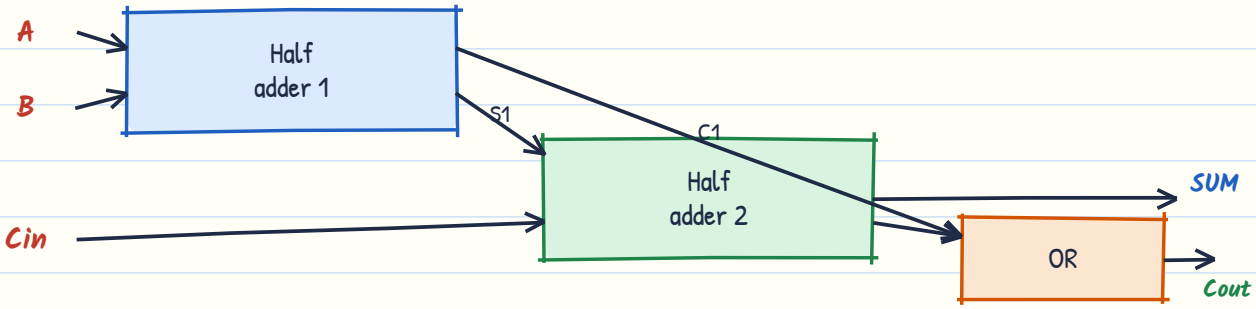


Fig 14: Full adder = 2 half adders + OR gate

- Sum = A (+) B (+) Cin Cout = AB + Cin(A (+) B)

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Decoder

A *decoder* converts an n-bit code into one of 2^n output lines (only one output active). Use: address decoding, memory chip select.

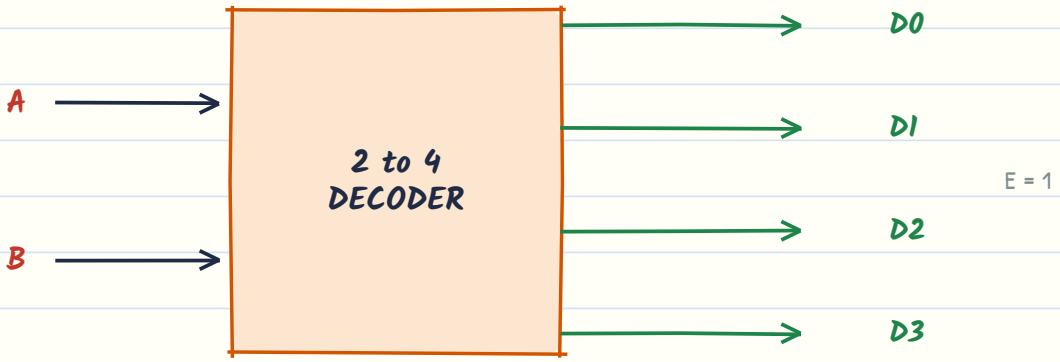


Fig 15: 2-to-4 decoder

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0

1	0	0	0	1	0
1	1	0	0	0	1

Each output is a ***minterm***: D0 = A'B', D1 = A'B, D2 = AB', D3 = AB. A ***3-to-8 decoder*** has 3 inputs and 8 outputs; two 3-to-8 decoders can build a 4-to-16 decoder (using the enable pin).

Multiplexer (MUX) - data selector

A ***multiplexer*** selects ***one of many inputs*** and sends it to a single output using select lines. 2^n inputs need n select lines.

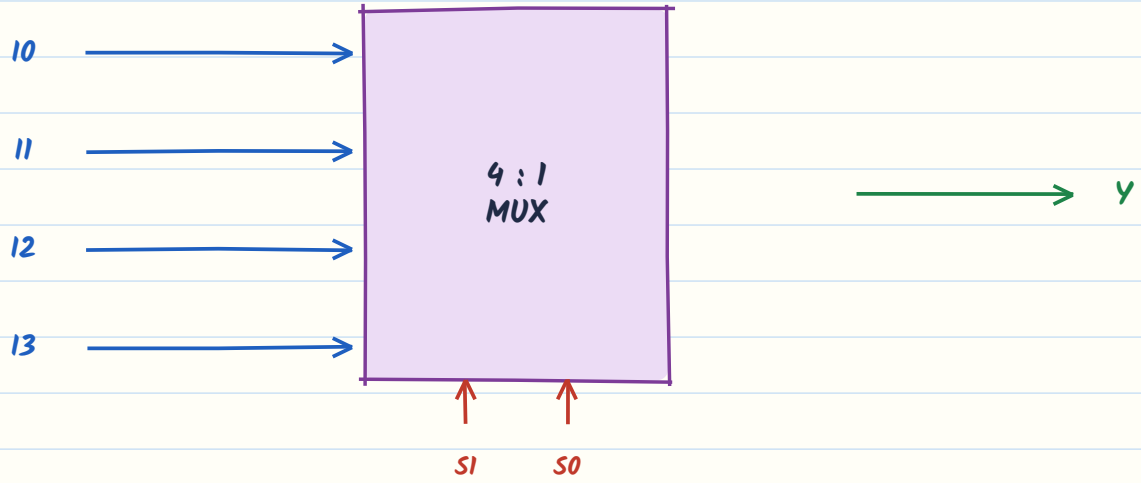


Fig 16: 4-to-1 multiplexer

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

- ***Y = S1'S0' I0 + S1'S0 I1 + S1S0' I2 + S1S0 I3***
- Larger MUX from smaller: two 4:1 + one 2:1 = 8:1.
- A MUX can implement any Boolean function (use inputs as 0/1 constants).

Quick Revision & Exam Questions

Topic	Memory line
Conversions	Dec->Bin: divide by 2; Bin->Oct/Hex: group 3/4 bits
IEEE 754	1 | 8 | 23 bits, bias 127, hidden 1
2's complement	invert + 1; range -8 to +7 (4-bit)
Gray code	one bit changes at a time

Universal gates	NAND, NOR
De Morgan	(AB)'=A'+B'; (A+B)'=A'B'
Decoder / MUX	n -> 2^n lines / 2^n inputs -> 1 output

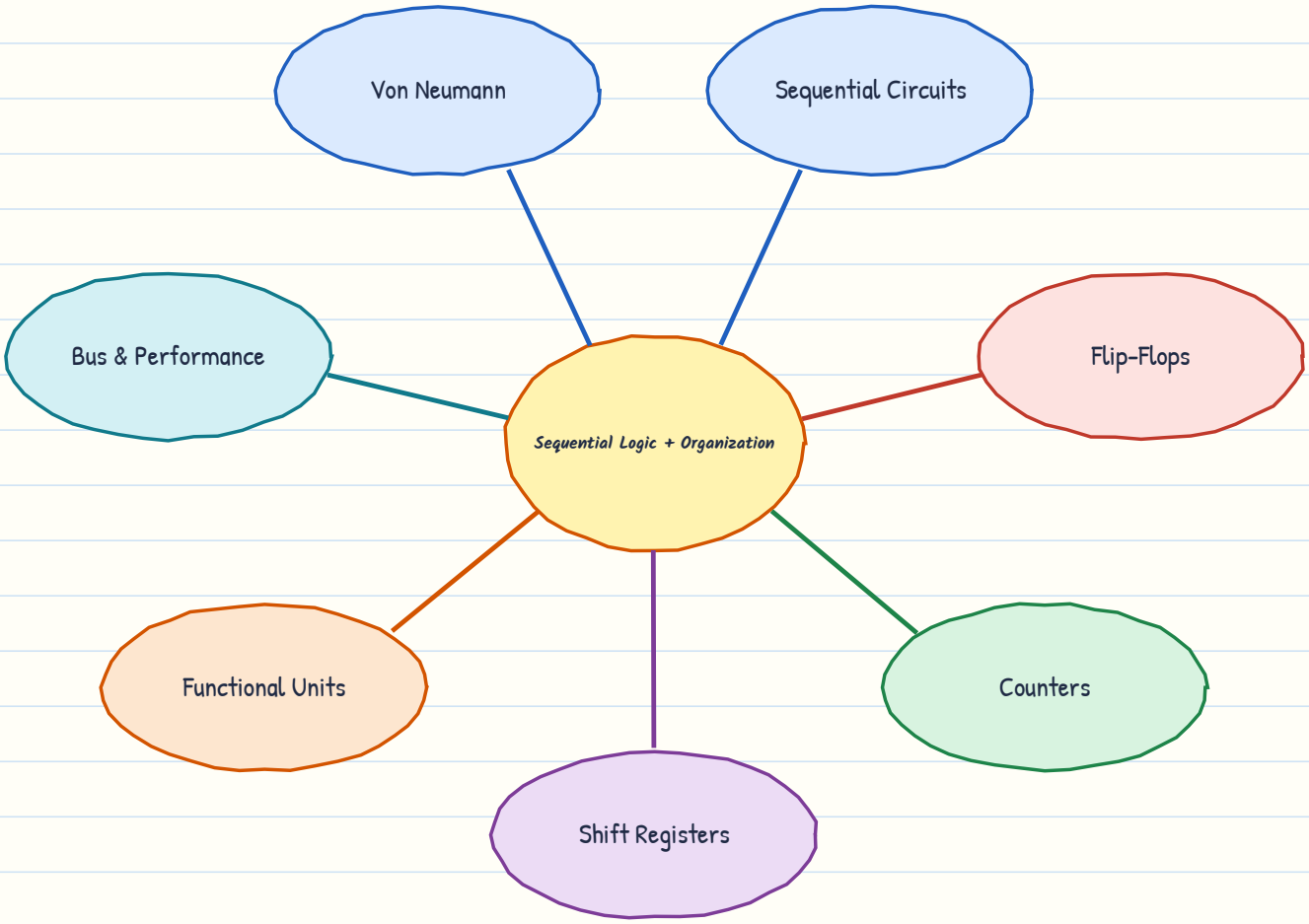
Likely exam questions

- Convert numbers between binary, octal, decimal and hexadecimal.
- Explain fixed and floating point representation (IEEE 754).
- Explain signed number representations; subtract using 2's complement.
- Explain binary codes: BCD, Excess-3, Gray, ASCII.
- Why are NAND and NOR universal? Realize AND, OR, NOT using NAND.
- Simplify a 3/4-variable function with K-map.
- Design a full adder; explain decoder and multiplexer.

UNIT II

Sequential Circuits & Basic Structure of Computers

Latches | Flip-flops | Counters | Registers | Functional units | Buses | Von Neumann



Mind map: everything you will learn in this unit

How to use these notes

Tip: for flip-flops learn the characteristic table + the block diagram together; for computer structure just remember the block diagrams.

I. Sequential Circuits

In a ***sequential circuit*** the output depends on the ***present inputs AND the past state*** – so it needs ***memory***. Combinational circuits have no memory.

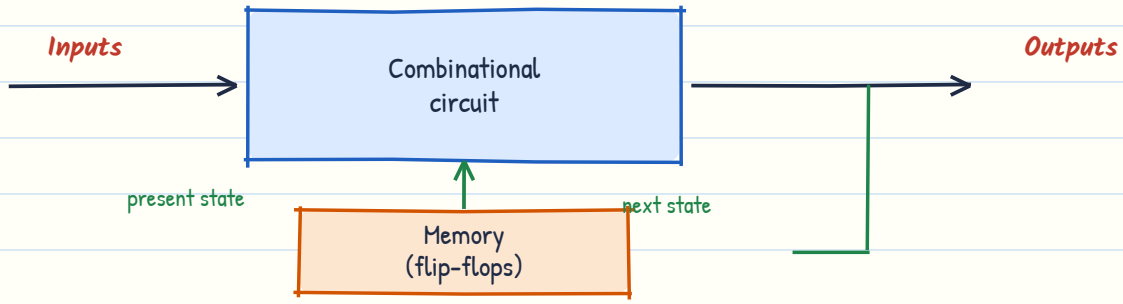


Fig 1: Sequential circuit = combinational logic + memory (feedback)

	Combinational	Sequential
Depends on	present input only	input + past state
Memory	no	yes (flip-flops)
Clock	not needed	usually clocked
Examples	adder, MUX, decoder	counter, register

- ***Synchronous:*** state changes only on a ***clock*** pulse (flip-flops).
- ***Asynchronous:*** state changes whenever inputs change (latches).

Latches – SR latch (the basic memory cell)

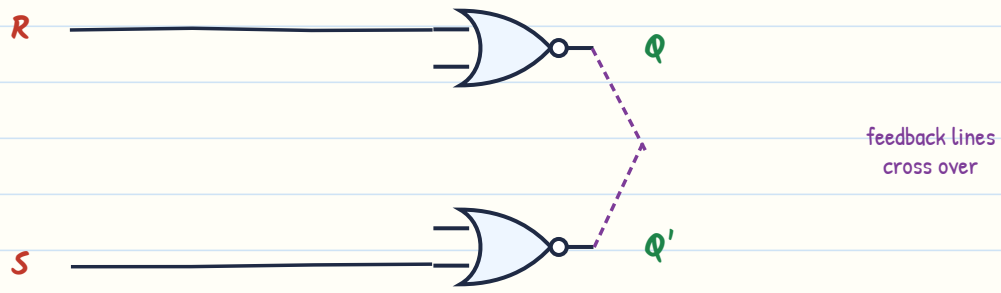


Fig 2: SR latch using two cross-coupled NOR gates

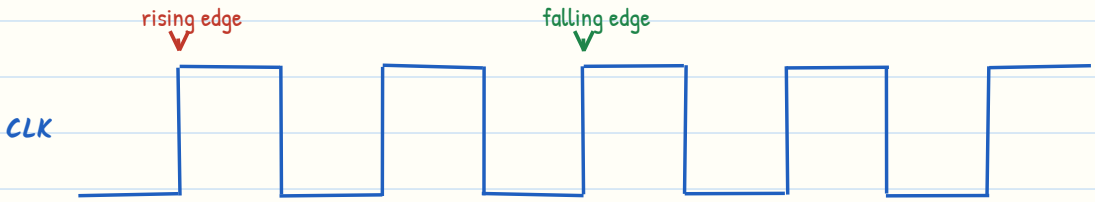
S	R	Q (next)	Action
0	0	no change	memory (hold)
0	1	0	RESET
1	0	1	SET
1	1	?	INVALID (not used)

Latch vs flip-flop

A latch is level-sensitive (changes while enable is high). A flip-flop is EDGE-triggered: it changes only at the clock edge (rising or falling) - more reliable.

2. Flip-Flops

A ***flip-flop (FF)*** is a 1-bit memory element that changes state only at a ***clock edge***. n flip-flops store n bits and give 2^n states.



FF samples input at the edge, holds value until the next edge

Fig 3: Clock signal and edge triggering

The four types of flip-flops

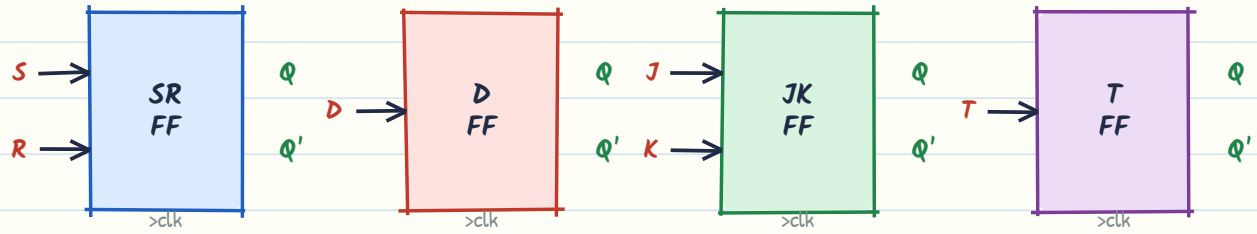


Fig 4: Symbols of SR, D, JK and T flip-flops

Type	Characteristic table	Equation
SR	00 hold, 01 reset, 10 set, 11 invalid	Q+ = S + R'Q
D	Q+ = D (copies input)	Q+ = D
JK	00 hold, 01 reset, 10 set, 11 toggle	Q+ = JQ' + K'Q
T	T=0 hold, T=1 toggle	Q+ = T xor Q

JK flip-flop (most versatile)

J	K	Q(t+1)	Meaning
0	0	Q	no change
0	1	0	reset
1	0	1	set
1	1	Q'	TOGGLE (SR's invalid state fixed)



Fig 5: JK flip-flop made from gates + master-slave idea

Concept

Race-around: in a plain level-triggered JK with J=K=1, the output toggles repeatedly while the clock is high. Cure: master-slave or edge-triggered JK.

Excitation tables (used in design)

Q	Q+	S R	D	J K	T
0	0	0 X	0	0 X	0
0	1	1 0	1	1 X	1
1	0	0 1	0	X 1	1
1	1	X 0	1	X 0	0

Excitation table answers: "to go from present state Q to next state Q+, what inputs are needed?" (X = don't care). Convert FFs: D from JK -> D = JQ'+K'Q; T from JK -> J=K=T.

3. Registers & Shift Registers

A *register* is a group of flip-flops storing a binary word (n FFs = n-bit register) - e.g. 8 D-flip-flops with a common clock make an 8-bit register.

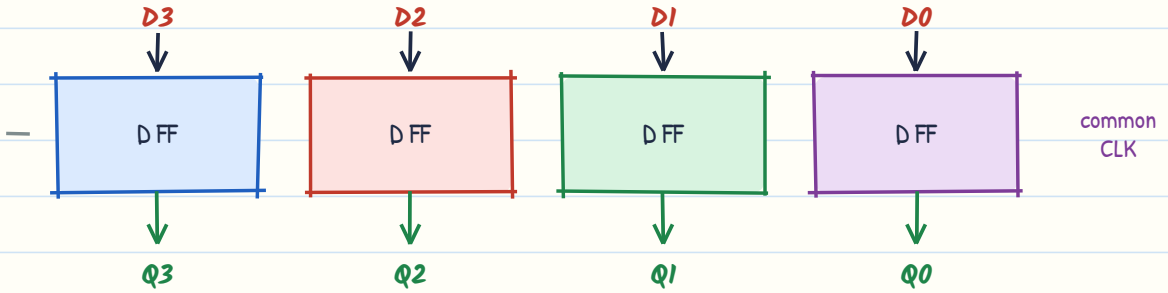
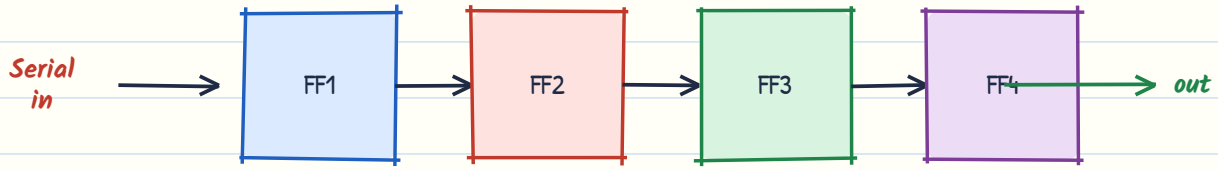


Fig 6: 4-bit parallel-load register (D flip-flops)

Shift register

A register that **shifts its bits one position** every clock pulse. Built from D flip-flops where Q of one feeds D of the next.



all FFs get the SAME clock: data moves right 1 place per pulse

Fig 7: 4-bit serial-in serial-out (SISO) shift register

Type	Input	Output	Use
SISO	serial	serial	delay line
SIPO	serial	parallel	serial -> parallel (receiver)
PISO	parallel	serial	parallel -> serial (transmitter)
PIPO	parallel	parallel	storage register
Universal	both, left/right	both	bidirectional register

Shifting example (SISO, input 1011, MSB first)

Clock	FF1	FF2	FF3	FF4
0 (reset)	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	0	1	0
4	1	1	0	1

- **Ring counter:** last FF output fed back to first FF - a single 1 circulates (n states).
- **Johnson (twisted-ring) counter:** inverted output fed back - 2n states.

4. Counters

A **counter** is a register that goes through a prescribed sequence of states on clock pulses. A counter with n flip-flops has up to **2^n states (modulus)**; mod-N counter counts N states.

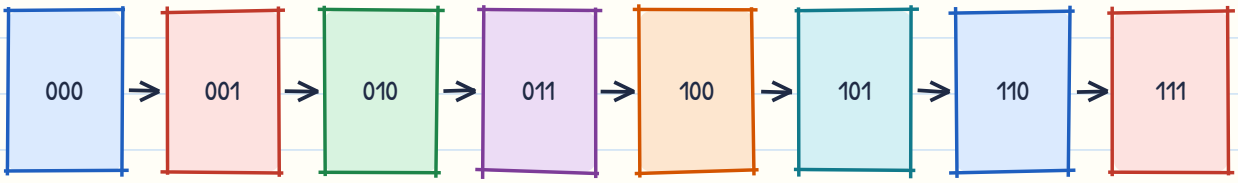
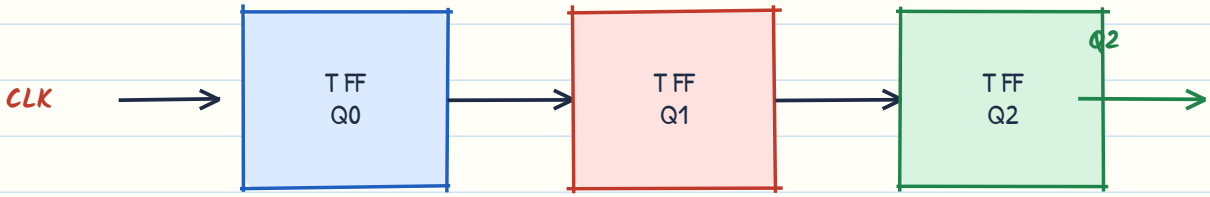


Fig 8: 3-bit binary up-counter state sequence

	Asynchronous (ripple)	Synchronous
Clock	only first FF gets clock	all FFs get same clock
Speed	slow (delay adds up)	fast
Circuit	simple	more gates
Glitches	yes	no

Ripple counter (asynchronous)



output of each FF is the clock of the next -> each stage divides frequency by 2

Q0 = CLK/2 Q1 = CLK/4 Q2 = CLK/8

Fig 9: 3-bit ripple counter using T flip-flops (T = 1)

- Count is read as Q2 Q1 Q0 (Q0 = LSB).
- **Down counter:** take clock from Q' instead of Q.
- **Mod-N (e.g. decade, mod-10) counter:** use a NAND gate to detect state N and clear (reset) all FFs.

Synchronous counter

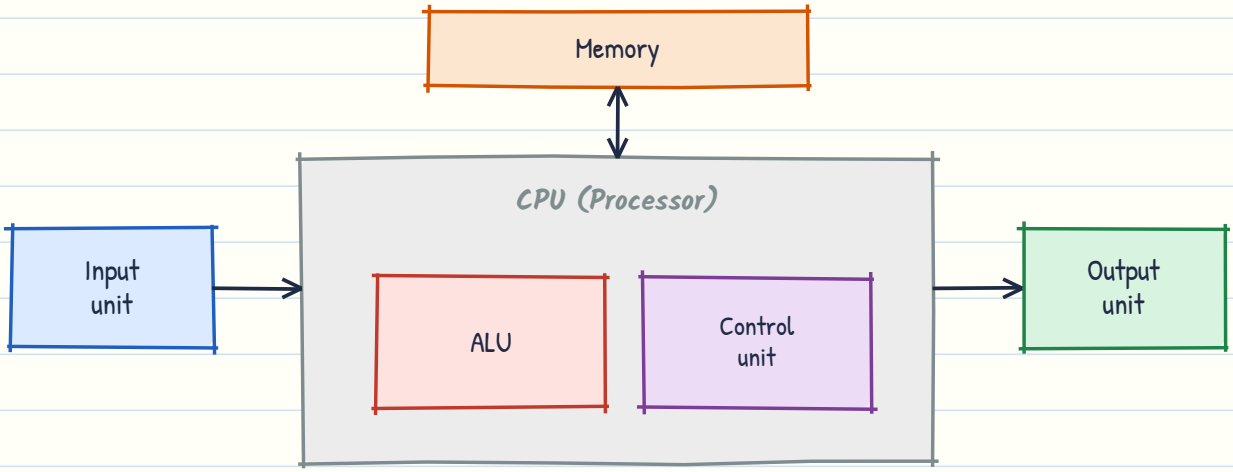
Every FF is driven by the **same clock**; toggle inputs are set by logic: T0 = 1, T1 = Q0, T2 = Q0.Q1 (a FF toggles only when all lower bits are 1). No ripple delay -> used for high speed.

5. Basic Structure of Computers

Computer types

Type	Description	Example
Embedded	built into devices, fixed task	washing machine, car ECU
Personal	single user desktop / laptop	PC, notebook
Workstation	powerful, graphics/engineering	CAD systems
Server	serves many users over network	web / database server
Supercomputer	extreme speed, parallel, science	weather forecasting
Mainframe	huge transaction processing	banks, airlines

Functional units



Secondary storage (disk, SSD): permanent, large, slow

Fig 10: Functional units of a computer

- **Input unit:** keyboard, mouse - converts data to binary.
- **Memory:** primary (RAM) for running programs; secondary for permanent storage.
- **ALU:** arithmetic and logic operations.
- **Control unit:** fetches instructions, decodes them and generates control signals for timing.
- **Output unit:** monitor, printer.

Basic operational concepts – registers and instruction cycle

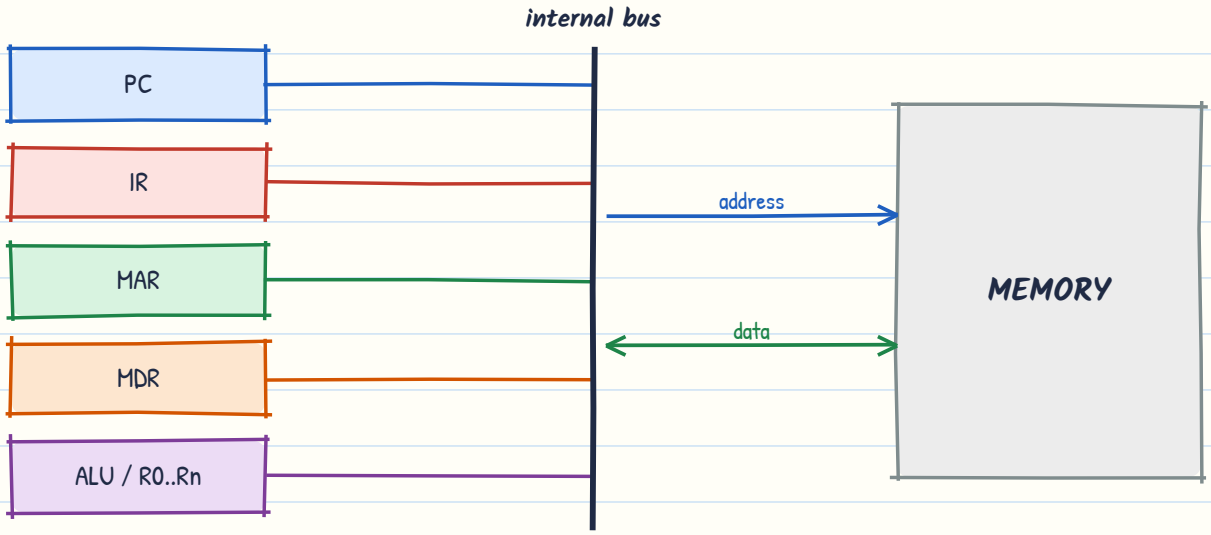


Fig 11: CPU registers connected to memory through the bus

Register	Full form	Job
PC	Program Counter	address of next instruction
IR	Instruction Register	holds current instruction
MAR	Memory Address Register	address to read/write
MDR	Memory Data Register	data being transferred
R0-Rn	General registers	temporary data
ACC / SP	Accumulator / Stack Ptr	results / stack top

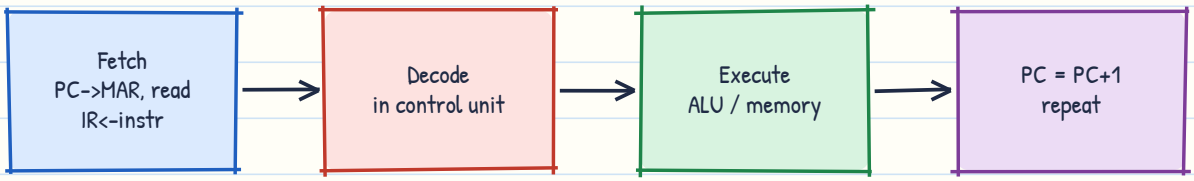


Fig 12: Instruction cycle

Bus structures

A ***bus*** is a set of shared wires that carries information between units. Three kinds of lines:

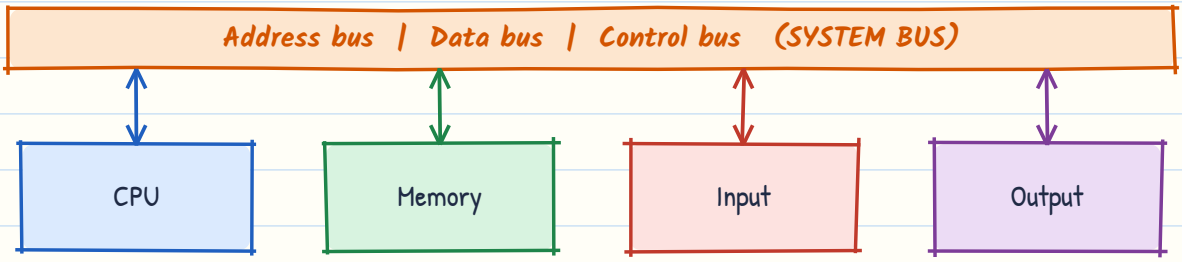


Fig 13: Single bus structure

Bus	Direction	Carries
Address	one way (CPU -> memory/IO)	location address
Data	two way	actual data / instructions
Control	both	read, write, clock, interrupt signals

- **Single bus:** cheap but only one transfer at a time (bottleneck).
- **Multiple bus:** several transfers at once, faster, costly.

Software

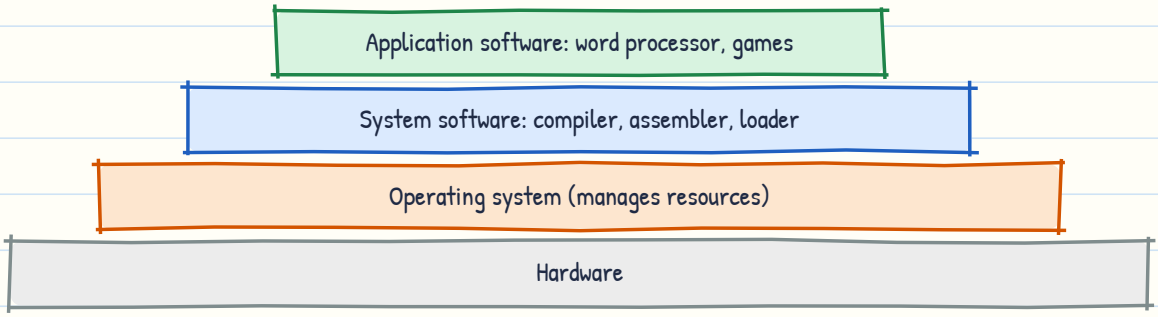


Fig 14: Layers of software

- **System software:** OS, compilers, assemblers, device drivers.
- **Application software:** user programs.
- **Compiler** translates the whole program; **interpreter** line by line; **assembler** converts assembly to machine code.

Performance

Measured by how fast a program executes. **Basic performance equation:**

$$T = (N \times S) / R$$

- T = program execution time, N = number of instructions, S = average **clock cycles per**

instruction (CPI), R = clock rate (Hz).

- To improve: reduce N (better compiler), reduce S (pipelining), increase R (faster technology).
- **MIPS** = instructions per second / 10^6; **MFLOPS** for floating point.

Multiprocessors and multicomputers

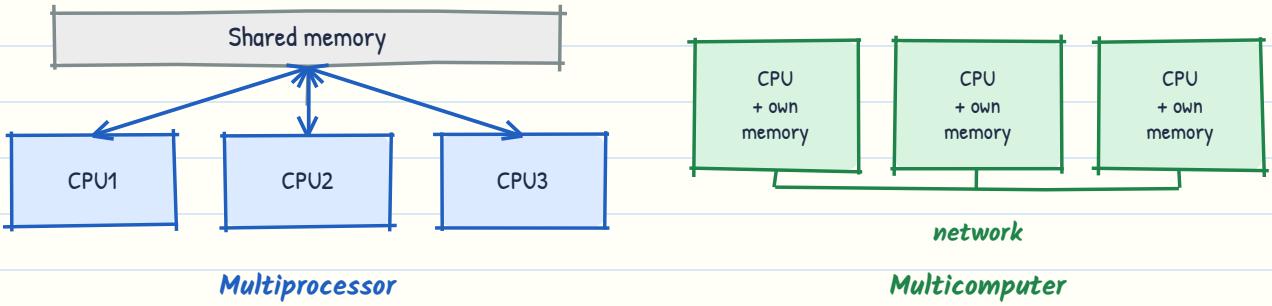


Fig 15: Multiprocessor (shared memory) vs multicomputer (message passing)

	Multiprocessor	Multicomputer
Memory	shared by all CPUs	each computer has private memory
Communication	via shared variables	message passing
Coupling	tightly coupled	loosely coupled

6. Computer Generations & Von Neumann Architecture

Gen	Period	Technology	Example
1st	1940-56	Vacuum tubes	ENIAC, UNIVAC
2nd	1956-63	Transistors	IBM 1401
3rd	1964-71	Integrated circuits (IC)	IBM 360
4th	1971-90s	Microprocessors (VLSI)	Intel 4004, PC
5th	present	ULSI, AI, parallel	AI systems

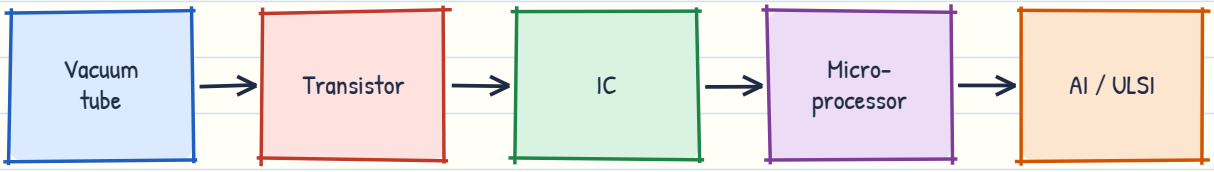


Fig 16: Evolution

Von Neumann architecture (stored-program concept)

Proposed by John von Neumann: ***instructions and data are stored together in the same memory***, and the CPU fetches them one by one.

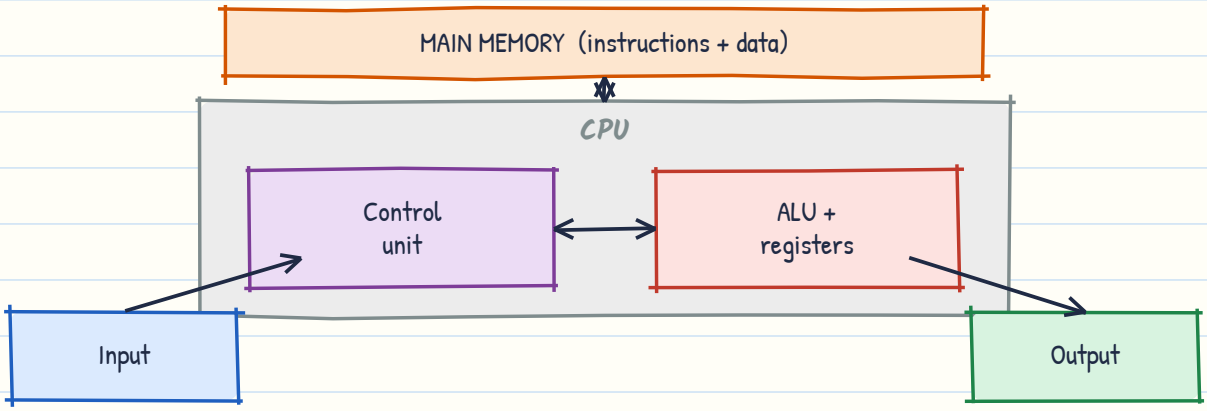


Fig 17: Von Neumann architecture

- One memory and one bus for both instructions and data -> ***Von Neumann bottleneck*** (cannot fetch both at once).
- ***Harvard architecture:*** separate memories and buses for instructions and data (faster; used in DSPs, cache split).

	Von Neumann	Harvard
Memory	single	separate
Bus	one	two
Speed	slower	faster
Cost	low	higher

Quick Revision & Exam Questions

Topic	Memory line
Sequential	output = f(input, past state); needs memory
JK 11	toggle
T FF	toggle when T=1; counters
n-bit counter	2^n states; ripple = async
PC/IR/MAR/MDR	next addr / instruction / address / data
T = N x S / R	performance equation
Von Neumann	one memory for program + data

Likely exam questions

- Differentiate combinational and sequential circuits.
- Explain SR, D, JK and T flip-flops with characteristic tables.
- Explain a shift register and its types; draw a 4-bit SISO register.
- Design / explain a 3-bit ripple counter with timing.
- Explain functional units of a computer with a block diagram.
- Explain bus structures and the basic performance equation.
- Explain Von Neumann architecture; compare with Harvard.
- Write about generations of computers.

COMPUTER ORGANIZATION

Subject Code : _____

UNIT – III

Computer Arithmetic & Processor Organization

Syllabus

Computer Arithmetic: Addition and Subtraction of Signed Numbers, Design of Fast Adders, Multiplication of Positive Numbers, Signed-operand Multiplication, Fast Multiplication, Integer Division, Floating-Point Numbers and Operations.

Processor Organization: Fundamental Concepts, Execution of a Complete Instruction, Multiple-Bus Organization, Hardwired Control and Microprogrammed Control.

Index

Topic	Page
1. Addition & Subtraction of Signed Numbers	2 – 3
2. Design of Fast Adders	4 – 5
3. Multiplication of Positive Numbers	6 – 7
4. Signed-operand Multiplication (Booth)	8 – 9
5. Fast Multiplication	10 – 11
6. Integer Division	12 – 13
7. Floating-Point Numbers & Operations	14 – 15
8. Processor Fundamentals & Instruction Execution	16 – 17
9. Multiple-Bus Organization	18
10. Hardwired Control	19
11. Microprogrammed Control	20

1. Addition & Subtraction of Signed Numbers

Representation of signed numbers (n bits): the MSB is the sign bit (0 = positive, 1 = negative). Three methods:

Method	Negative of +5 (4-bit)	Range	Zero
Sign-magnitude	1101	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	two (±0)
1's complement	1010	$-(2^{n-1}-1) \dots +(2^{n-1}-1)$	two (±0)
2's complement	1011	$-2^{n-1} \dots +(2^{n-1}-1)$	one (0)

2's complement (most used)

- Reason: one adder works for both positive and negative numbers; only one zero.
- To negate: invert all bits (1's complement) and add 1. Ex: +5 = 0101 → 1010 → +1 = 1011 = −5.
- Value = $-b_{n-1}\cdot 2^{n-1} + \Sigma\, b_i \cdot 2^i$ $(i = 0 \dots n-2)$.
- Sign extension: copy the sign bit to the left to widen a number (1011 → 11111011).

Addition rules (2's complement)

- Add the two n-bit numbers bit by bit, carries moving left.
- Discard the carry out of the MSB. No correction step is needed.
- Result is correct if no overflow occurs.

Ex 1: (+3) + (−5) = 0011 + 1011 = 1110 = −2 ✔

Ex 2: (−3) + (−2) = 1101 + 1110 = 1 1011 → drop carry → 1011 = −5 ✔

Ex 3: (+6) + (+2) = 0110 + 0010 = 1000 → ?? overflow!

Subtraction & Overflow

Subtraction

$X - Y = X + (2\text{'s complement of } Y) = X + \bar{Y} + 1$. So the same adder is used: invert Y with XOR gates and apply 1 to the carry-in c_0.

Ex: (+5) − (+3): 0101 + (1100 + 1) = 0101 + 1101 = 1 0010

Discard carry → 0010 = +2 ✔

Overflow

Result lies outside the range of n bits $-2^{n-1} \ldots 2^{n-1}-1$.

- Occurs only when adding numbers of the SAME sign and the result has the OPPOSITE sign.
- (+ve) + (−ve) can never overflow.
- Detection: Overflow = $c_n \oplus c_{n-1}$ (carry into MSB XOR carry out of MSB).
- Or by sign bits: $V = x_{n-1}\, y_{n-1}\, \bar{s}_{n-1} + \bar{x}_{n-1}\, \bar{y}_{n-1}\, s_{n-1}$.

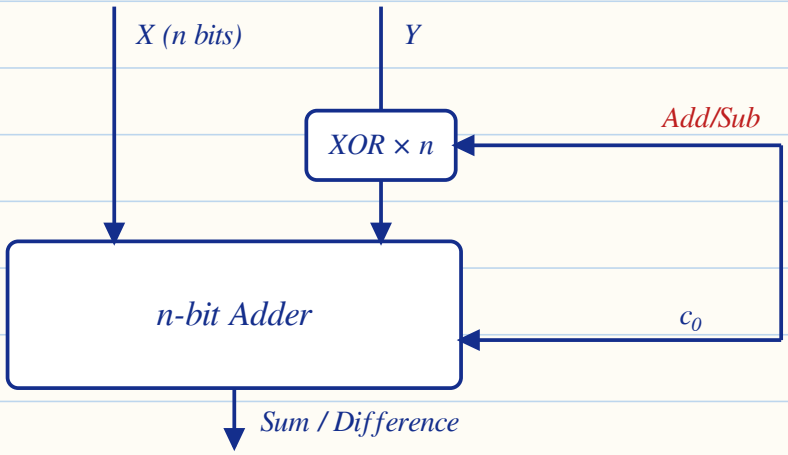
Ex: 0111 (+7) + 0101 (+5) = 1100 → carry into MSB = 1, carry out = 0 → Overflow = 1 ✔ (true sum 12 > 7).

Add / Subtract unit (logic diagram)

Add/Sub = 0 → X + Y

Add/Sub = 1 → X − Y

(Y inverted, c_0=1)



Add/Sub control line selects operation; the XOR gates pass Y unchanged (0) or complement it (1). Overflow flag generated from $c_n \oplus c_{n-1}$.

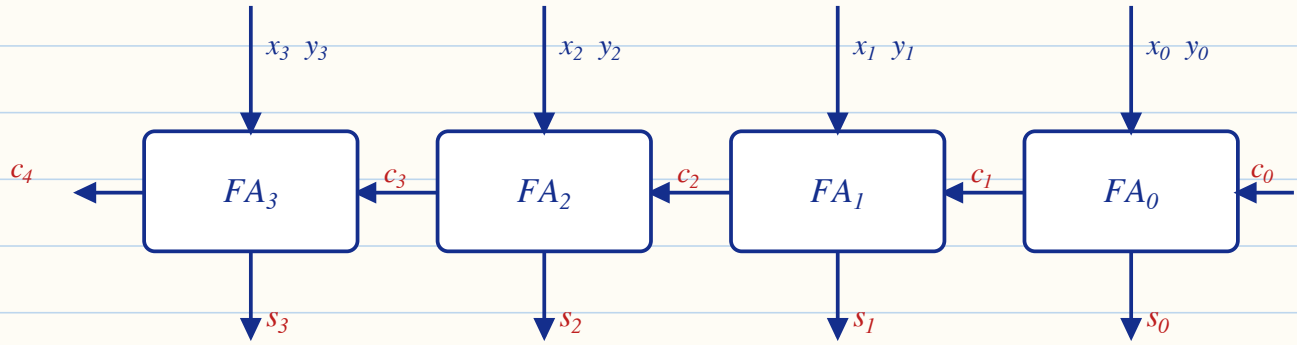
2. Design of Fast Adders

Full Adder (FA)

$s_i = x_i \oplus y_i \oplus c_i$ $\quad$ $c_{i+1} = x_i\, y_i + x_i\, c_i + y_i\, c_i$

n-bit Ripple-Carry Adder

n full adders in series; carry c_{i+1} of one stage is the carry-in of the next.



- *Delay: each FA needs 2 gate delays for carry → total ≈ 2n gate delays (last sum ≈ 2n+1).*
- *Problem: slow for large n (32/64-bit) because every carry waits for the previous stage.*
- *Solution: compute carries in advance → Carry-Lookahead Adder.*

Carry-Lookahead Adder (CLA) – derivation

Rewrite the carry: $c_{i+1} = x_i y_i + (x_i + y_i)\, c_i \;=\; G_i + P_i\, c_i$

- *Generate* $G_i = x_i\, y_i$ *(stage creates a carry on its own).*
- *Propagate* $P_i = x_i + y_i$ *(stage passes an incoming carry).*

Expanding the recurrence repeatedly:

$$c_1 = G_0 + P_0 c_0$$

$$c_2 = G_1 + P_1 G_0 + P_1 P_0 c_0$$

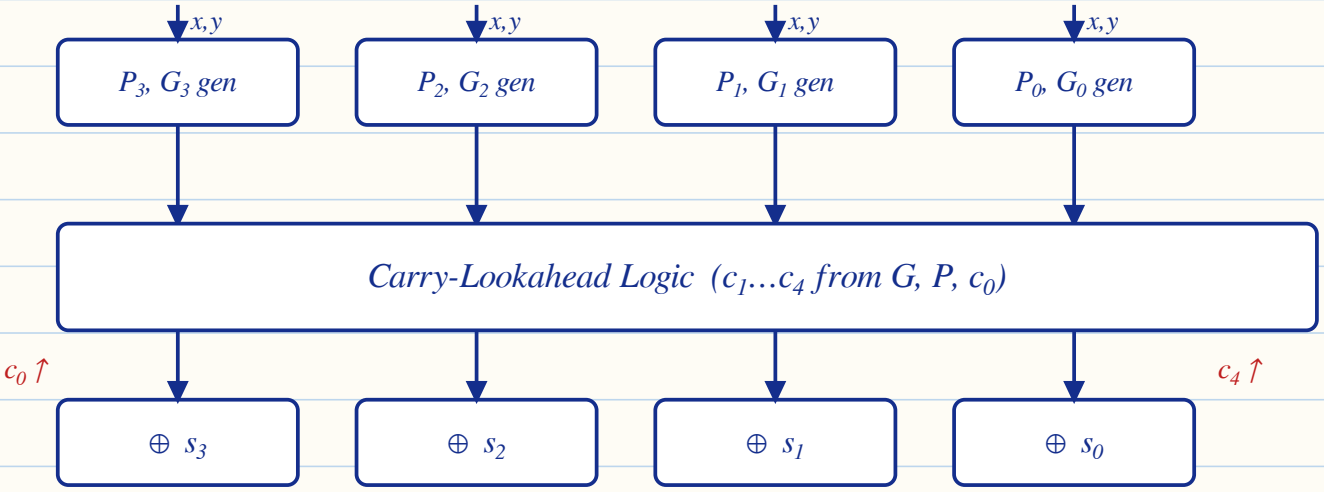
$$c_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0$$

$$c_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0$$

Carry-Lookahead Adder (contd.)

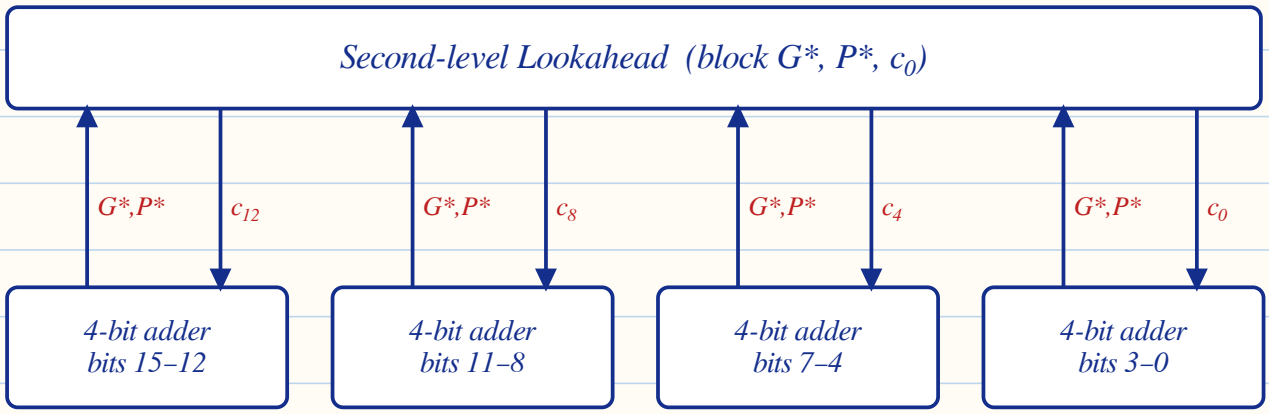
All carries depend only on x, y and c_0, so they are produced in PARALLEL by one AND-OR network. Sum: $s_i = P_i' \oplus c_i$, with $P_i' = x_i \oplus y_i$.

4-bit CLA block



- *Delay: PG generation 1 gate, carry logic 2 gates, sum XOR 1 gate → about 4 gate delays, independent of n for the block.*

Cascading blocks (16-bit / 32-bit adder)



- *Block generate $G^* = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$*
- *Block propagate $P^* = P_3P_2P_1P_0$*

Adder	Delay (n-bit)	Hardware
Ripple-carry	≈ 2n gate delays	Least
CLA (blocked)	≈ 8 for 32-bit	More

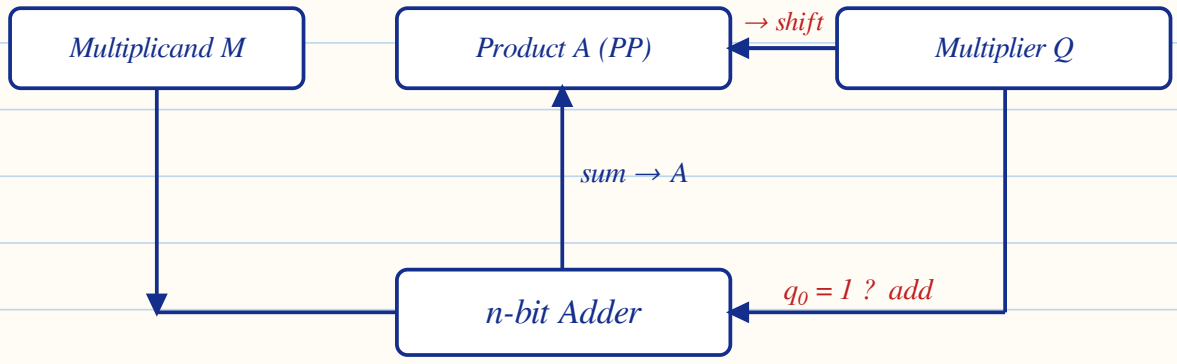
3. Multiplication of Positive Numbers

Multiplying an n-bit multiplicand M by an n-bit multiplier Q gives a 2n-bit product.

Paper-and-pencil method: for each multiplier bit q_i write M (if q_i=1) or 0 (if q_i=0), shifted left i places, and add all partial products.

```
    1101   (13)  Multiplicand M
×   1011   (11)  Multiplier Q
 ------
    1101
   1101
  0000
 1101
--------
10001111   (143)
```

Sequential (shift-and-add) multiplier



- *Registers: M (multiplicand), Q (multiplier), A (partial product, init 0), carry flip-flop C, counter n.*
- *Repeat n times: if $q_0 = 1$ then $A = A + M$; then shift C,A,Q right one bit.*
- *At the end the 2n-bit product is in A:Q. Time ≈ n additions.*

Worked Example & Array Multiplier

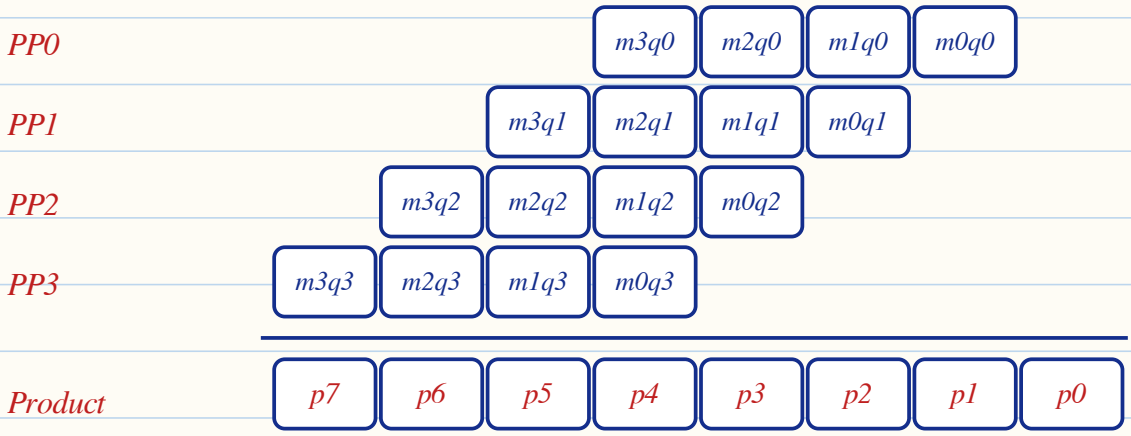
13 × 11, with M = 1101, Q = 1011, n = 4:

Step	Action	C	A	Q
Init	—	0	0000	1011
1	q0=1: A=A+M, shift	0	0110	1101
2	q0=1: A=A+M, shift	0	1001	1110
3	q0=0: shift only	0	0100	1111
4	q0=1: A=A+M, shift	0	1000	1111

Product = A:Q = 1000 1111 = 143 ✔

Array (combinational) multiplier

AND gates give partial products $PP_i = M \cdot q_i$;
FA array adds column-wise (diagonal ripple).



- *Each cell = AND gate + full adder; n×n array of cells (n^2 cells).*
- *Delay ≈ (2n − 1) FA delays (longest path along diagonal and bottom row) → very regular layout, good for VLSI.*
- *Faster than sequential (no clock steps) but needs much more hardware.*

Important Questions

Q *Explain the sequential multiplier with 13 × 11.*

Q *Write a note on the array multiplier.*

4. Signed-operand Multiplication

Shift-and-add fails for negative 2's-complement numbers (sign bit has negative weight). Simple fix: multiply magnitudes and fix the sign — slow. Better: Booth's algorithm.

Booth's Algorithm – idea

A run of 1s in the multiplier, …0 1 1 1 1 0… $= 2^{k+1} - 2^{j}$. So instead of adding M for every 1 bit, do ONE subtraction at the start of the run and ONE addition just after its end.

0 0 1 1 1 1 0 = $+2^5 - 2^1$ → (+32 − 2 = 30)

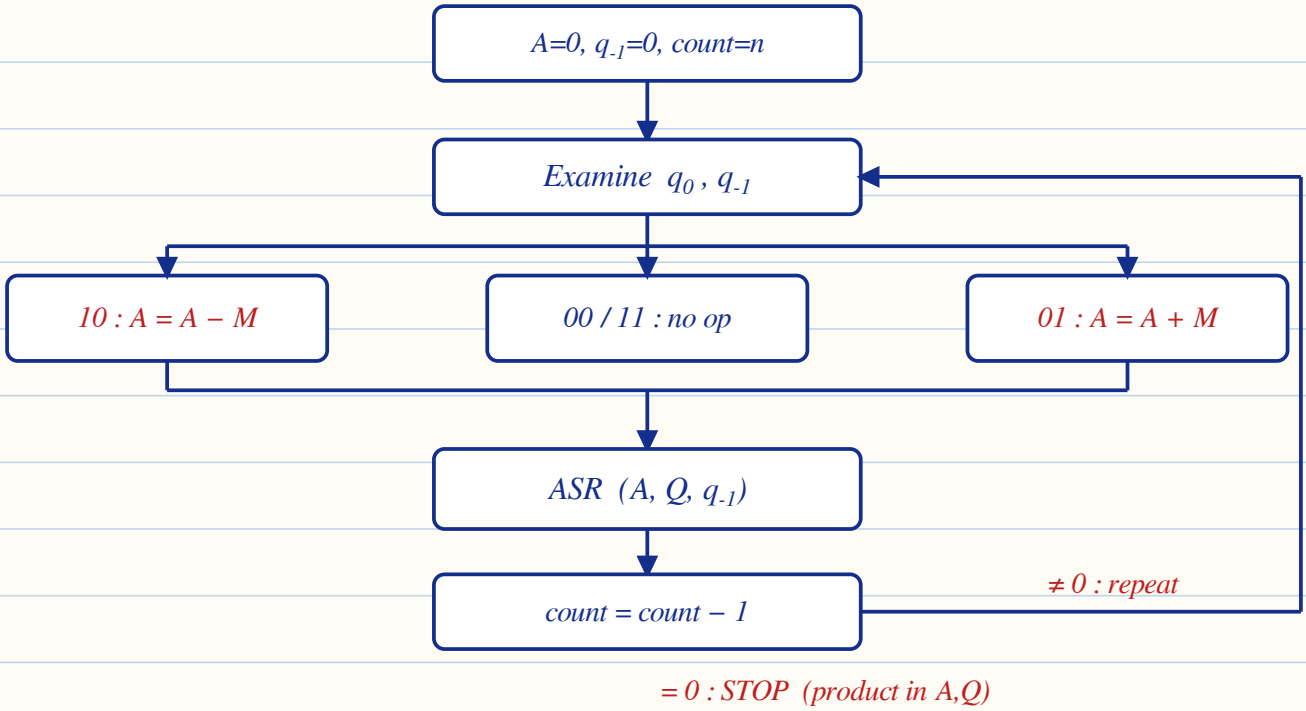
Fewer add/sub operations when the multiplier has long runs of 1s.

Recoding table (scan q_0 and q_{-1}, q_{-1} = 0 initially)

q0	q–1	Action on A	Booth bit
0	0	none, shift	0
0	1	A = A + M	+1
1	0	A = A − M	−1
1	1	none, shift	0

- *After each step perform an ARITHMETIC right shift of A, Q, q_{-1} (sign bit of A is preserved).*
- *Works for +ve and −ve multiplicand and multiplier — no correction required.*
- *Worst case: alternating bits 0101… → n operations; best: all 0s / 1s.*
- *Advantage: handles signs automatically; avg. speed gain for runs of 1s.*

Booth's Algorithm – Flowchart & Example



Ex: (+7) × (–3), n = 4: M = 0111, –M = 1001, Q = 1101

Step	q0 q–1	Operation	A	Q	q–1
Init			0000	1101	0
1	1 0	A=A−M, ASR	1100	1110	1
2	0 1	A=A+M, ASR	0001	1111	0
3	1 0	A=A−M, ASR	1101	0111	1
4	1 1	ASR only	1110	1011	1

Result A:Q = 1110 1011 = –21 ✔ (7 × –3 = –21)

Important Questions

Q *Explain Booth's algorithm with a flowchart; multiply +7 × −3.*

Q *What are the advantages of Booth's algorithm?*

5. Fast Multiplication

Two techniques speed up multiplication: (a) reduce the NUMBER of partial products (bit-pair recoding), (b) add partial products FASTER (carry-save addition).

(a) Bit-pair recoding of the multiplier (Booth radix-4)

Group the Booth-recoded multiplier into pairs → number of summands is halved (n/2). Each pair (q_{i+1}, q_i) with q_{i-1} selects one multiple of M:

q(i+1) q(i) q(i–1)	Multiple of M	Booth pair
0 0 0	0	0
0 0 1	+ M	+1
0 1 0	+ M	+1
0 1 1	+ 2M	+2
1 0 0	– 2M	–2
1 0 1	– M	–1
1 1 0	– M	–1
1 1 1	0	0

- *Shifts by 2 bits per step; 2M is just M shifted left one place (no extra hardware).*

Ex: multiplier Q = 1010 (–6), q_{-1} = 0. Pair 1 = (q1,q0,q-1) = 1 0 0 → –2M. Pair 2 = (q3,q2,q1) = 1 0 1 → –M (weight 4). Total = –2M – 4M = –6M ✔

Carry-Save Addition (CSA) & Wallace Tree

Adding k partial products with ripple adders takes (k−1) full carry propagations. CSA avoids propagating carries: each FA column takes 3 input bits and gives a Sum bit and a Carry bit (kept SEPARATE, carry shifted one place left).

X = 1 0 1 1

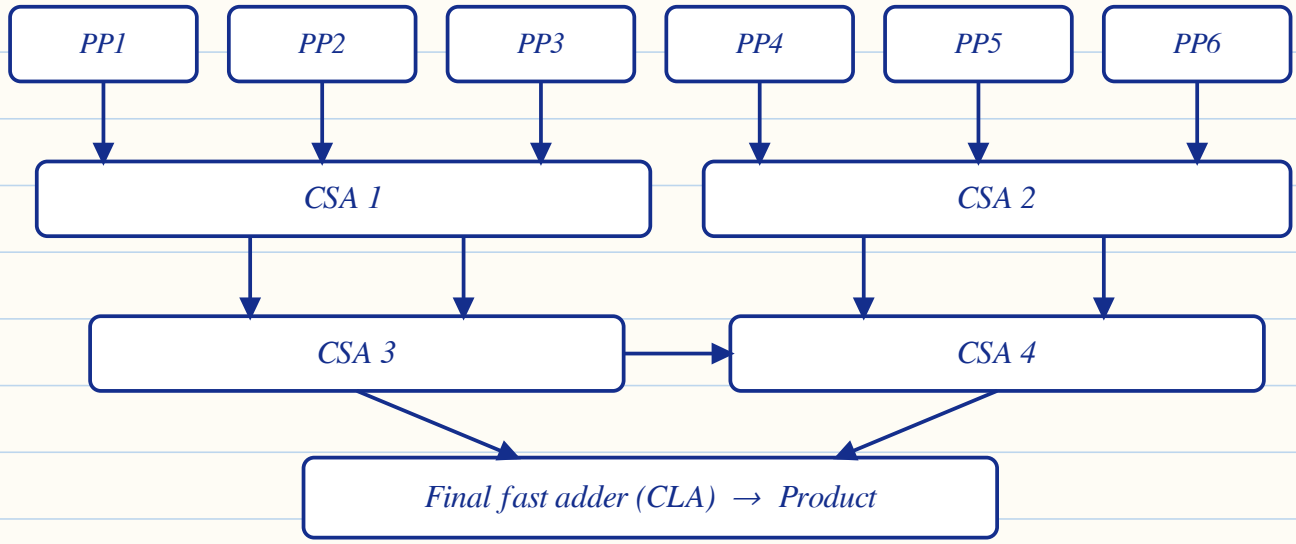
Y = 0 1 1 0

Z = 1 1 0 1

Sum = X⊕Y⊕Z (save)

Carry = majority, shifted left

Wallace / CSA tree

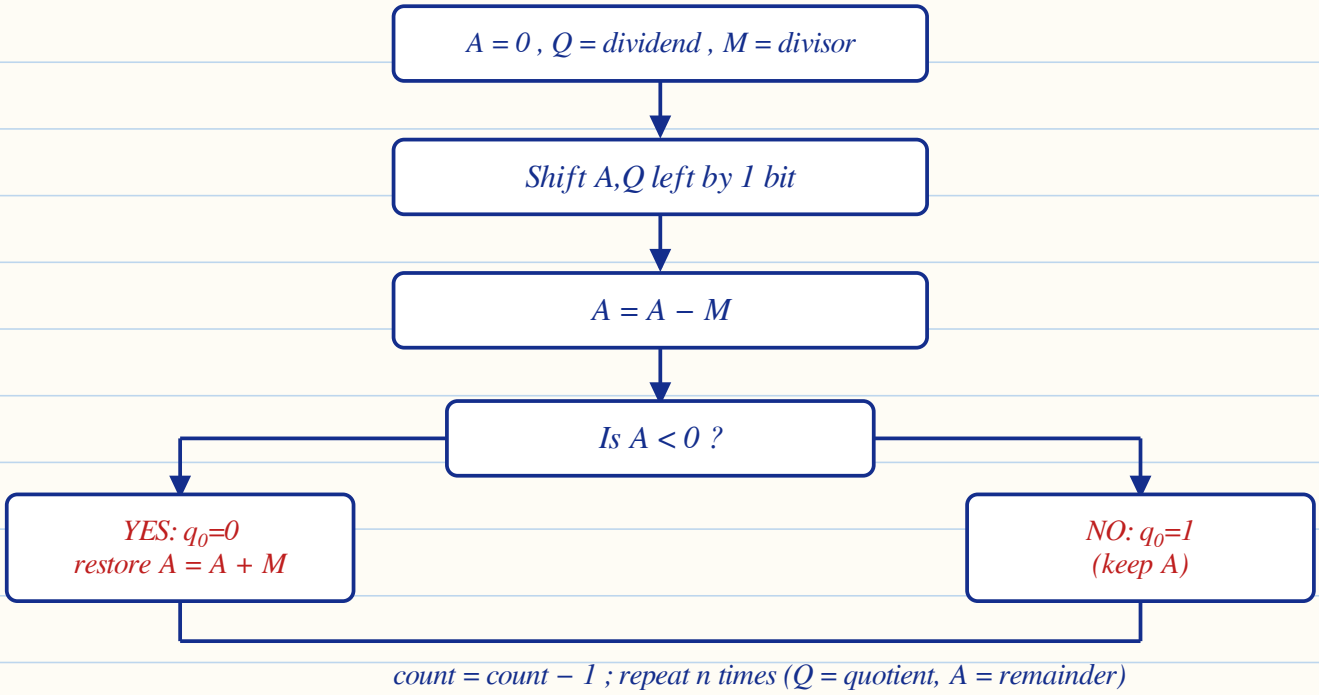


- *3 rows → 2 rows at every CSA level; with k PPs the number of levels ≈ $\log_{1.5}(k/2)$.*
- *Only the FINAL two rows are added with a fast (CLA) adder → delay ∝ log n instead of n.*
- *Combine with bit-pair recoding: n/2 partial products, then CSA tree → fastest practical multiplier.*
- *Disadvantage: irregular wiring, more area than array multiplier.*

6. Integer Division

Dividend = Quotient × Divisor + Remainder. Registers: A (n+1 bits, remainder, init 0), Q (dividend, then quotient), M (divisor), count n.

Restoring division



- *Each cycle: shift left A:Q, subtract M from A; if result is negative put back M (restore) and q_0=0, else q_0=1.*
- *Needs an extra add (restore) → slow when many restores.*

Non-restoring division

Avoids the restore step. Rule per cycle:

- *If A ≥ 0: shift left A:Q, then A = A − M.*
- *If A < 0: shift left A:Q, then A = A + M.*
- *After the operation: A ≥ 0 → q_0=1, else q_0=0.*
- *At the end, if A < 0 add M once to correct the remainder.*

Division – Worked Examples

Restoring: 8 ÷ 3 (n = 4, M = 0011)

Step	A:Q after shift	A−M	Result	A	Q
Init				0000	1000
1	0001 000_	1110 (<0)	q0=0, restore	0001	0000
2	0010 000_	1111 (<0)	q0=0, restore	0010	0000
3	0100 000_	0001 (≥0)	q0=1	0001	0001
4	0010 001_	1111 (<0)	q0=0, restore	0010	0010

Quotient = 0010 = 2, Remainder = 0010 = 2 ✔ (8 = 3 × 2 + 2)

Comparison

Point	Restoring	Non-restoring
Operations / cycle	sub + (maybe) add	one add OR sub
Speed	slower	faster
Final correction	none	add M if A<0
Hardware	simple	slightly more control

- Overflow / divide-by-zero must be detected before division starts (divisor = 0 → error flag).
- Signed division: divide magnitudes, quotient sign = sign(dividend) ⊕ sign(divisor), remainder takes the dividend's sign.

Important Questions

Q Explain restoring and non-restoring division with an example.

Q Differentiate restoring vs non-restoring division.

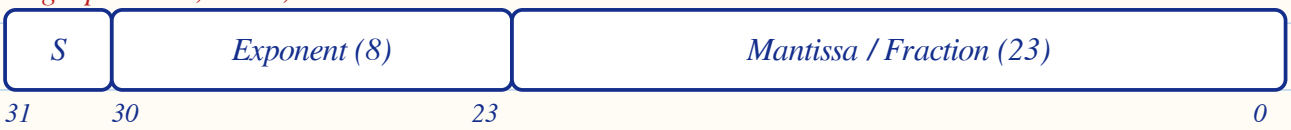
7. Floating-Point Numbers & Operations

Scientific form to represent very large and very small numbers: $\pm 1.M \times 2^{E-bias}$.

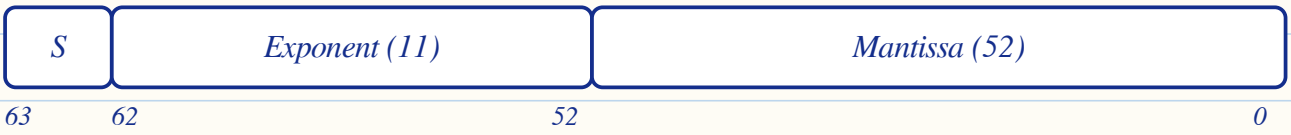
A floating-point number stores Sign, Exponent, Mantissa.

IEEE 754 formats

Single precision, 32 bit, bias = 127



Double precision, 64 bit, bias = 1023



- *Normalized: leading 1 is implied (hidden bit) → 1.M.*
- *Exponent stored in excess (biased) form: E = true exponent + 127 (single) or + 1023 (double).*
- *Value* $= (-1)^S \times 1.M \times 2^{E-127}$.

Example: represent 12.375

$12.375 = 1100.011_2 = 1.100011 \times 2^3 \rightarrow S = 0, E = 3+127 = 130 = 10000010, M = 10001100\ldots0$

IEEE word: 0 | 10000010 | 10001100000000000000000 = 0x41460000

Special values (single precision)

Exponent	Mantissa	Meaning
0000 0000	0	± 0
0000 0000	≠ 0	denormalized
1111 1111	0	± ∞
1111 1111	≠ 0	NaN (not a number)

Floating-Point Operations

Add / Subtract

Floating-point add/subtract : 5 steps



- *1) Compare exponents, difference $d = |E_1 - E_2|$. 2) Shift mantissa of the smaller number right d places.*
- *3) Add / subtract mantissas (sign handling). 4) Normalize result (shift, adjust exponent). 5) Round.*

Ex: $1.5 + 0.25 = 1.1\times2^0 + 1.0\times2^{-2}$. Align: 0.01×2^0. Add: $1.1 + 0.01 = 1.11\times2^0 = 1.75$

✔

Multiply / Divide

- *Multiply: $E = E_1 + E_2 - bias$; $M = M_1 \times M_2$; $S = S_1 \oplus S_2$; normalize, round.*
- *Divide: $E = E_1 - E_2 + bias$; $M = M_1 \div M_2$; $S = S_1 \oplus S_2$; normalize, round.*

Guard bits & rounding

Extra bits kept during alignment: Guard (g), Round (r), Sticky (s = OR of all bits shifted beyond r). Rounding modes:

- *Chop (truncate) • Round to nearest even (default) • Toward $+\infty$ • Toward $-\infty$*

Exceptions

- *Exponent overflow (result too big → ∞), underflow (too small → 0/denormal), inexact, invalid (0/0, $\infty-\infty$ → NaN), divide-by-zero.*

Important Questions

Q *Explain IEEE 754 format; represent 12.375.*

Q *Explain floating-point addition steps.*

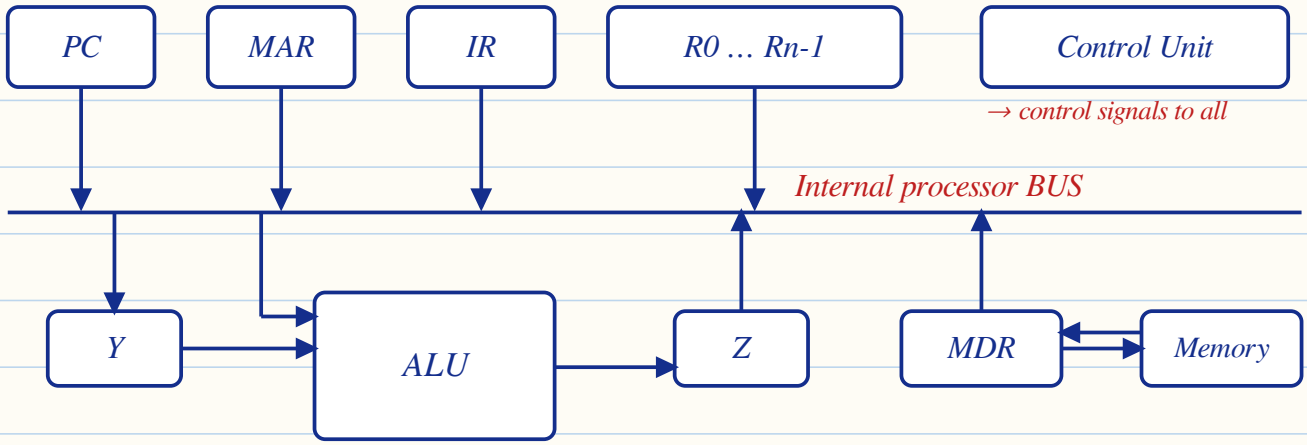
8. Processor Organization – Fundamental Concepts

Processor = Datapath (registers + ALU + buses) + Control unit. It executes instructions in fetch–decode–execute cycles.

Main registers

- *PC – address of next instruction; IR – holds current instruction; MAR – memory address; MDR – memory data;*
- *R0…Rn-1 – general registers; Y, Z – temporary registers around the ALU.*

Single-bus datapath



Only ONE register can put data on the bus per clock step (others only read).

Register transfers use gating: R_{out} puts a register on the bus, R_{in} loads it from the bus (control signals). Example: PC_{out}, MAR_{in}.

- *Clock period ≥ time for one bus transfer + ALU + setup.*
- *Memory read: MAR_{in}, Read, wait for MFC (Memory Function Completed), then MDR_{inE} → IR.*

Important Questions

Q *Explain the single-bus datapath of a processor.*

Q *Write the control sequence for fetching an instruction.*

Execution of a Complete Instruction

Instruction cycle = Fetch + Decode + Execute + Store. Example: ADD R1, R2 (single bus), and a branch.

Fetch (common to all)

Step	Control signals	Action
1	PC_out, MAR_in, Read, Select4, Add, Z_in	MAR ← PC; Z ← PC+4
2	Z_out, PC_in, Y_in, WMFC	PC ← Z; wait for memory
3	MDR_out, IR_in	IR ← instruction
4	—	decode opcode

Execute: ADD R1, R2 (R1 ← R1 + R2)

Step	Control signals	Action
4	R1_out, Y_in	Y ← R1
5	R2_out, Select Y, Add, Z_in	Z ← Y + R2
6	Z_out, R1_in, End	R1 ← Z

Execute: unconditional BRANCH offset

Step	Control signals	Action
4	Offset_field_of_IR_out, Add, Z_in	Z ← PC + offset
5	Z_out, PC_in, End	PC ← target

- Conditional branch: step 4 tests condition codes; if false, the End signal is generated early.
- Load/Store need extra steps for memory access of the operand through MAR/MDR.

Important Questions

Q Write the control steps of ADD R1,R2 and a branch.

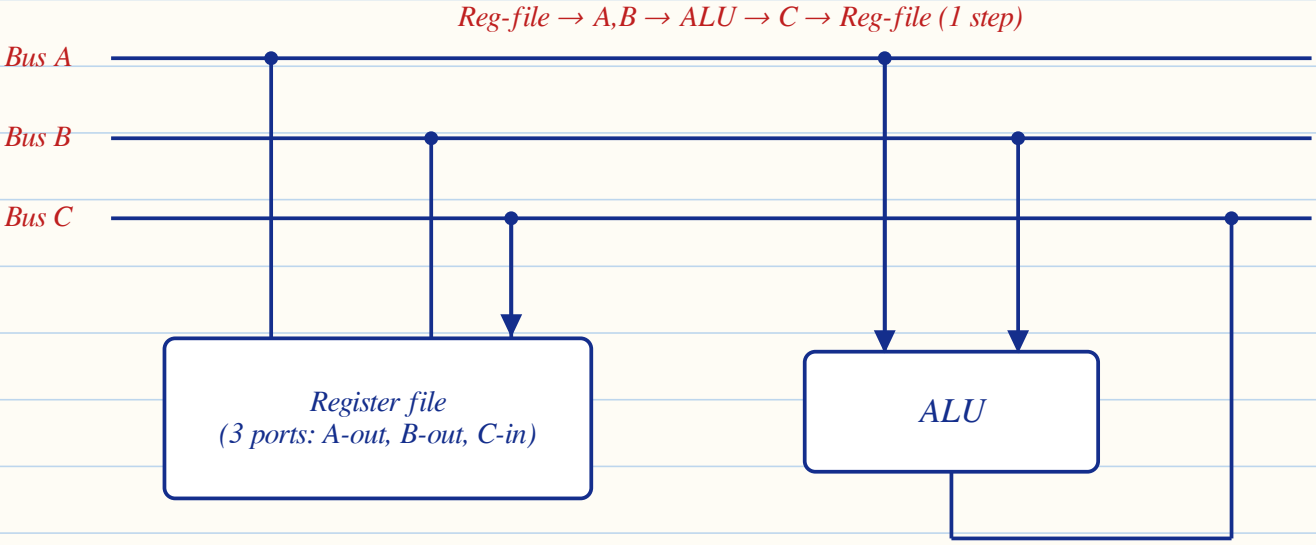
Q Explain how the multiple-bus organization increases speed.

9. Multiple-Bus Organization

In a single-bus processor only one data transfer per clock → many steps per instruction.

Providing several buses allows parallel transfers.

Three-bus organization



- *Bus A and Bus B carry the two ALU operands out of a register file with two read ports; Bus C carries the result back to the register file (one write port).*
- *Registers Y and Z are NOT needed; a separate incrementer updates PC (PC ← PC + 4) in parallel.*
- *Add R4, R5, R6 : R4_out→A, R5_out→B, ALU Add, R6_in← C → just ONE step.*

Comparison

Feature	Single bus	Three bus
Transfers / cycle	1	up to 3
Steps for ADD	≈ 3	1
Hardware cost	low	high
Performance	lower	higher

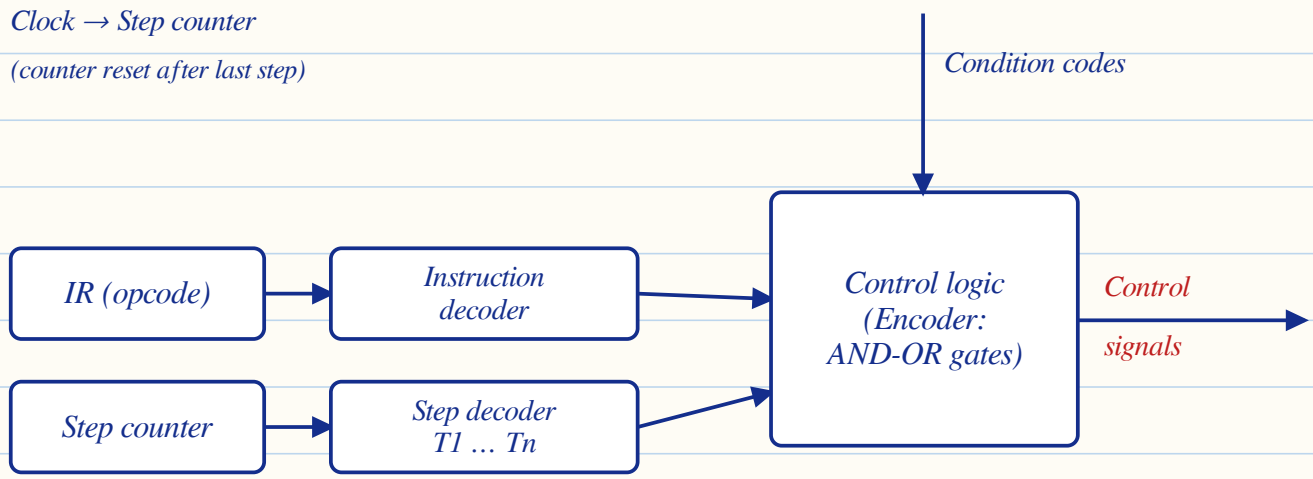
Important Questions

Q *Explain the three-bus organization with a diagram.*

Q *Compare single-bus and multiple-bus processors.*

10. Hardwired Control

Control signals are produced by a fixed sequential logic circuit (gates, decoders). Fastest control; used in RISC processors.



Inputs and outputs

- *Inputs: control step counter (decoded to $T_1 \dots T_n$), instruction decoder ($INS_1 \dots INS_m$ from IR), condition codes, external inputs (interrupts).*
- *Outputs: control signals such as PC_{out}, MAR_{in}, Add, Z_{in}, End.*

Each signal = OR of the (step AND instruction) conditions where it is needed:

$$Z_{in} = T_1 + T_6 \cdot ADD + T_4 \cdot BR + \dots$$

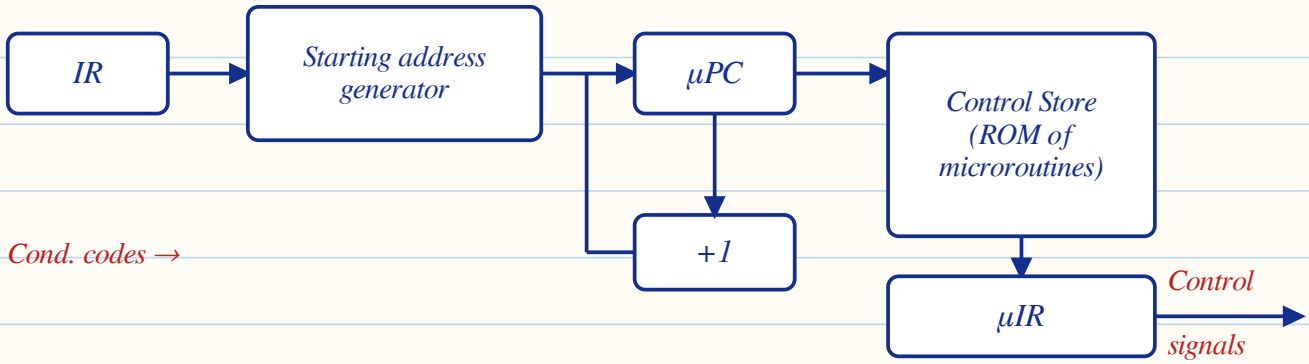
$$End = T_7 \cdot ADD + T_5 \cdot BR + (T_5 \cdot N + T_4 \cdot \bar{N}) \cdot BRN + \dots$$

- *Step counter is reset to T_1 by the End signal.*
- *Advantages: very fast; Disadvantages: hard to design / modify, complex for large instruction sets (CISC).*

11. Microprogrammed Control

Control signals are stored as control words (microinstructions) in a control store (ROM).

The set of microinstructions for one machine instruction = microroutine.



μPC points to next microinstruction; branch address loaded from μIR

Microinstruction formats

Horizontal (1 bit per signal, wide word)

Next addr	PCout	MARin	Read	Zin	Add	…	Yin

Vertical (encoded fields, decoder needed)

Next addr	F1 (4 bit)	F2 (3 bit)	F3 (2 bit)

- *Sequencing: μPC ← starting address from IR opcode; then μPC+1, or branch address (with conditions); fetch microroutine is common.*
- *Wide-branch addressing (bit-ORing) and prefetching of microinstructions improve speed.*

Hardwired	Microprogrammed
Fast, fixed logic	Slower, ROM lookup
Hard to modify	Easy to change/extend
Used in RISC	Used in CISC

UNIT - IV

THE MEMORY ORGANIZATION

Computer Organization - Handwritten Study Notes

Syllabus topics:

Basic Concepts, Semiconductor RAM Memories, Read-Only Memories, Speed-Size-Cost, Cache Memories, Performance Considerations, Virtual Memories, Memory Management Requirements, Secondary Storage

I. Basic Concepts

Memory is the place where **programs and data are stored** . The processor fetches instructions and data from memory and writes the results back into it.

- Memory is made of millions of **storage cells** ; each cell stores **1 bit** (0 or 1).
- Bits are grouped into **words** (word length = 8, 16, 32 or 64 bits). A word is the unit of transfer.
- Every word has a unique **address** . A k-bit address can reach 2^k locations: 16-bit address -> 2^16 = 64K words, 32-bit address -> 2^32 = 4G.
- Most machines are **byte-addressable** (each byte has its own address).

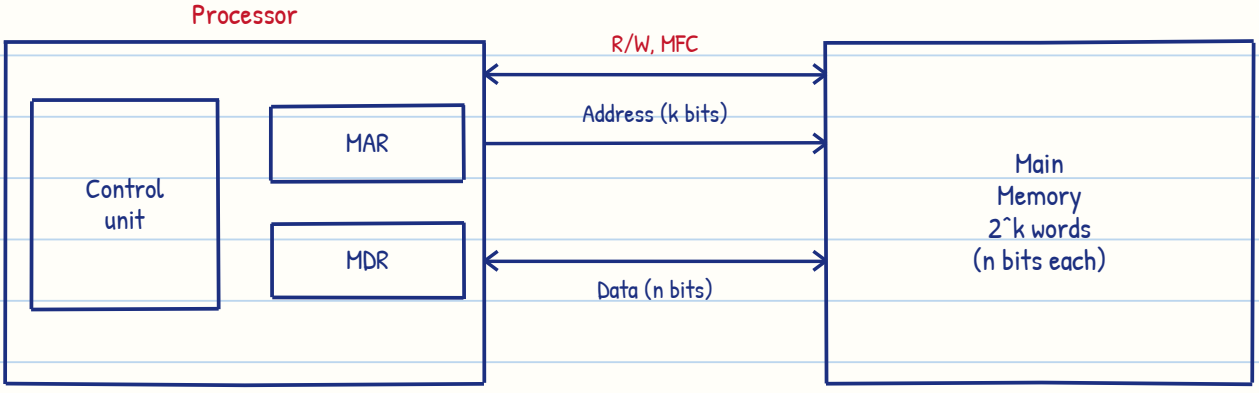


Fig 1: Connection of the processor and the memory

Important registers and signals

- **MAR** (Memory Address Register) holds the address of the location to be accessed.
- **MDR** (Memory Data Register) holds the data being read from / written to memory.
- **R/W** control line tells memory whether to Read or Write. **MFC** (Memory Function

Completed) is the reply signal from memory.

Read operation (step by step)

1. Processor loads the address into **MAR** ; it appears on the address lines.
2. Processor sets **R/W = 1** (Read) and starts the operation.
3. Memory places the word on the data lines and sets **MFC** .
4. Processor copies the data into **MDR** .

- **Write** is the same, but the data is first put in MDR and R/W = 0 (Write); memory stores it and sets MFC.

Key definitions

Memory access time = time between starting an operation (Read signal) and its completion (data available). **Memory cycle time** = minimum delay between two successive memory operations (always >= access time). **RAM** = Random Access Memory: any location is reached in the same time. **Bandwidth** = number of bits/bytes transferred per second.

2. Semiconductor RAM Memories

2.1 Internal organisation of a memory chip

Cells are arranged in a **rows x columns array** . All cells of a row form one word and share a **word line** . Each column has **bit lines** that connect to a **Sense/Write circuit** .

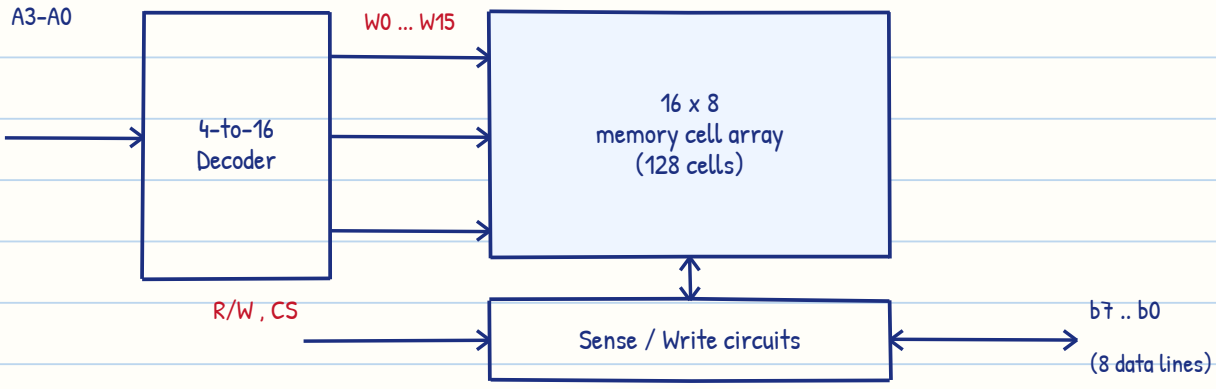


Fig 2: Organisation of a 16 x 8 memory chip

- 16 words x 8 bits = 128 cells. 16 words need **4 address bits** (A3-A0); the **decoder** activates exactly one word line.
- **R/W** selects the operation; **CS** (Chip Select) enables the chip when many chips are used.

- External pins: 4 address + 8 data + R/W + CS + 2 power = **16 pins** .
- Bigger chips save pins by organising the array as a square. Example **1K x 1** chip: 32 x 32 array, **5-bit row decoder** + **5-bit column selector** = 10 address bits.

2.2 Static RAM (SRAM)

SRAM cells hold their state **as long as power is ON** (no refresh needed). Memory that loses data when power is off is called **volatile** .

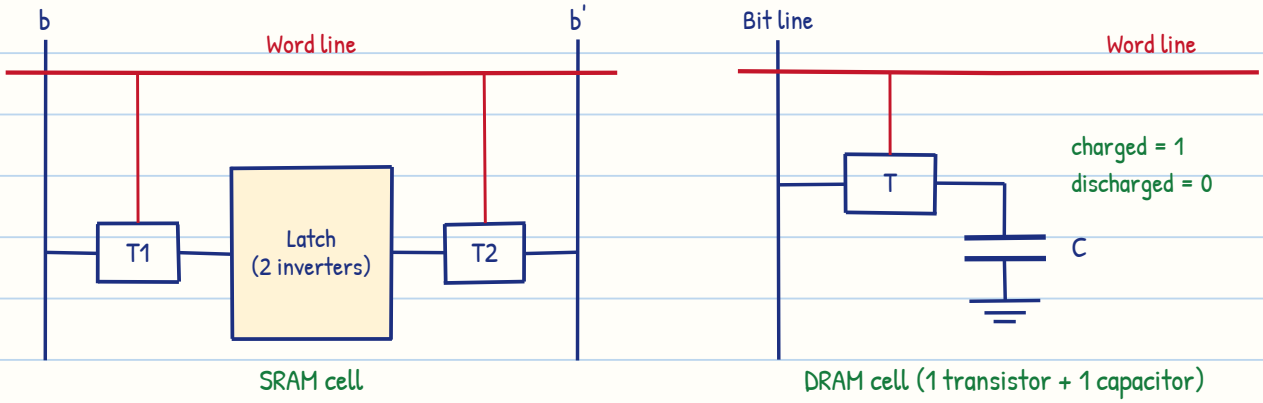


Fig 3: SRAM cell (left) and DRAM cell (right)

- The cell is a **latch** of two cross-coupled inverters. Two transistors **T1, T2** connect it to bit lines **b** and **b'** .
- **Read:** word line = 1 -> T1, T2 ON -> latch value appears on b and b' -> sense circuit reads it.
- **Write:** Sense/Write circuit puts the value on b and its complement on b', then word line = 1 -> latch is forced to the new state.
- CMOS SRAM cell uses **6 transistors** . Very **fast (a few ns)** and low power, but **expensive** and low density. Used for **cache memory** .

2.3 Dynamic RAM (DRAM)

- Cell = **1 transistor + 1 capacitor** . Charged capacitor = 1, discharged = 0. Much smaller and cheaper than SRAM, so used for **main memory** .
- The capacitor **leaks** its charge, so every row must be **refreshed** (re-written) every few milliseconds (typically 16-64 ms).
- **Read:** word line = 1 -> transistor ON -> charge goes to bit line -> sense amplifier detects it and **rewrites** the value (restore).
- **Write:** bit line is set to the required voltage, word line = 1, capacitor is charged or discharged.

Asynchronous DRAM

- Address is sent in two halves on the **same pins** : **row address** with **RAS** (Row Address

Strobe), then **column address** with **CAS** (Column Address Strobe). This saves pins.

- **Fast Page Mode:** once a row is selected, many columns of that row can be read by changing only the column address and pulsing CAS -> fast block transfers.

Synchronous DRAM (SDRAM)

- Operations are **synchronised with a clock** from the processor/bus. Row is read into a latch (row buffer) and consecutive columns come out in **bursts** (e.g. 4 or 8 words) with a single address.
- **DDR SDRAM** (Double Data Rate) transfers data on **both rising and falling clock edges** -> twice the bandwidth. DDR2, DDR3, DDR4... are later generations with higher speed and lower voltage.
- **Latency** = time to get the first word; **Bandwidth** = words per second. Both decide memory performance.

2.4 Building large memories from chips

Example: build a **2M x 32** memory (2M = 2^21 words) using **512K x 8** chips.

- Chips in one row = 32 / 8 = **4** (to make a 32-bit word). Rows = 2M / 512K = **4** . Total chips = 4 x 4 = **16** .
- 21-bit address: the lower **19 bits** (2^19 = 512K) go to every chip; the upper **2 bits** go to a **2-to-4 decoder** that selects one row of chips (Chip Select).
- Dynamic memories also need a **memory controller** : it multiplexes the address into row/column parts, generates RAS/CAS and does the **refresh** automatically.
- DRAM chips are sold as small boards called **SIMMs / DIMMs** (Single/Dual In-line Memory Modules).

Feature	SRAM	DRAM
Cell	6 transistors (latch)	1 transistor + 1 capacitor
Refresh	Not needed	Needed every few ms
Speed	Very fast	Slower
Cost / density	Costly, low density	Cheap, high density
Used for	Cache	Main memory

3. Read-Only Memories (ROM)

ROM is **non-volatile** : contents stay even when power is off. Normal operation only **reads** it. It stores the **bootstrap loader / firmware** that starts the computer.

- **Storing a bit:** a transistor connects the bit line to ground -> bit = 0; no connection -> bit = 1. The word line activates the row and the bit line shows 0 or 1.

Type	How programmed / erased	Remarks
ROM (mask)	Programmed by the manufacturer during fabrication	Cheap only for large volumes; cannot change
PROM	User programs once by blowing fuses (high current)	One-time programmable
EPROM	Erased by ultraviolet light (whole chip), then reprogrammed	Has a transparent quartz window
EEPROM	Erased and written electrically, cell by cell	No need to remove the chip; slow, costly
Flash	Electrically erased in blocks, written in bytes/pages	High density, low power; used in USB drives, memory cards, SSD

Note: Flash memory has a limited number of write/erase cycles but no moving parts. NAND flash (high density) is used in SSDs; NOR flash is used for firmware.

4. Speed, Size and Cost

The **ideal memory** would be very fast, very large and very cheap - but this is impossible. **SRAM** is fast but costly, **DRAM** is cheaper but slower, **disk** is very cheap per bit but extremely slow. So computers use a **memory hierarchy** .

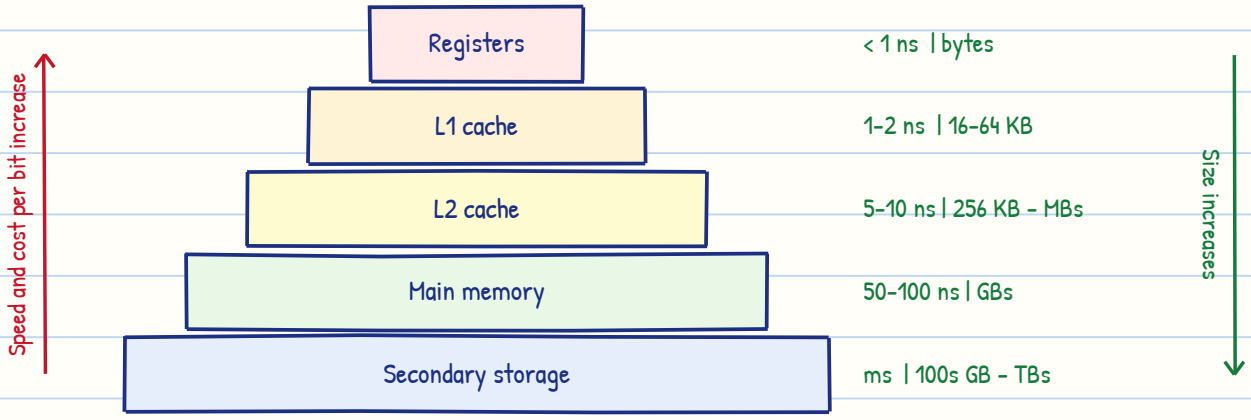


Fig 4: Memory hierarchy (typical approximate values)

- Moving **down** the hierarchy: size increases, cost per bit decreases, speed decreases.
- It works because of **locality of reference** - programs use the same or nearby data again and again, so most accesses are served by the fast upper levels.

- Result: the processor sees a memory that is **almost as fast as cache** and **as large and cheap as disk** .

5. Cache Memories

A **cache** is a small, very fast SRAM placed between the processor and main memory. It keeps copies of the **most recently used** instructions and data so the processor rarely waits for slow main memory.

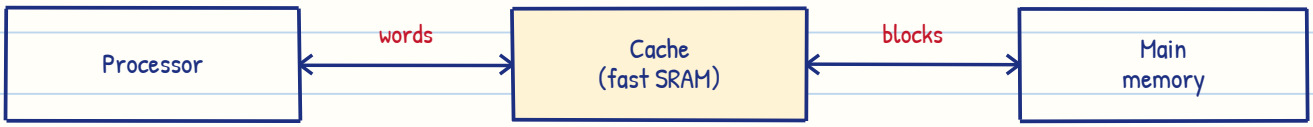


Fig 5: Cache between processor and main memory

Locality of reference

- **Temporal locality:** a recently used item will be used again soon (e.g. loop instructions).
- **Spatial locality:** items near a used address will be used soon (e.g. array elements). So a whole **block (cache line)** of consecutive words is brought in on a miss.

Basic terms

- **Hit:** requested word is in the cache. **Miss:** it is not. **Hit rate** = hits / total accesses; **miss rate** = 1 - hit rate.
- **Block (line):** group of consecutive words moved between memory and cache (e.g. 16 words).

Read and write handling

- **Read hit:** word is sent to the processor straight from cache - main memory is not involved.
- **Read miss:** the whole block is loaded from main memory. With **load-through (early restart)** the needed word is forwarded to the processor as soon as it arrives, without waiting for the full block.
- **Write hit** has two protocols:
  - **Write-through:** cache and main memory are updated together. Simple, memory is always correct, but many slow memory writes.
  - **Write-back (copy-back):** only cache is updated and the block is marked with a **dirty (modified) bit** . Memory is updated only when the block is replaced. Fewer memory writes.
- **Write miss:** write-through -> write directly to main memory. Write-back -> first load

the block into cache, then update it.

Mapping functions (where does a memory block go in the cache?)

Example used for all three methods

Main memory = 64K words (**16-bit address**). Block = 16 words -> 4096 memory blocks. Cache = 2K words = **128 blocks** .

(a) Direct mapping

- Memory block **j** can go to **only one** cache block: **j mod 128** . (Blocks 0, 128, 256 ... all compete for cache block 0.)
- Address is divided as **Tag (5) | Block (7) | Word (4)** : Block picks the cache block, Word picks the word inside it, **Tag** tells which of the 32 competing memory blocks is stored now.

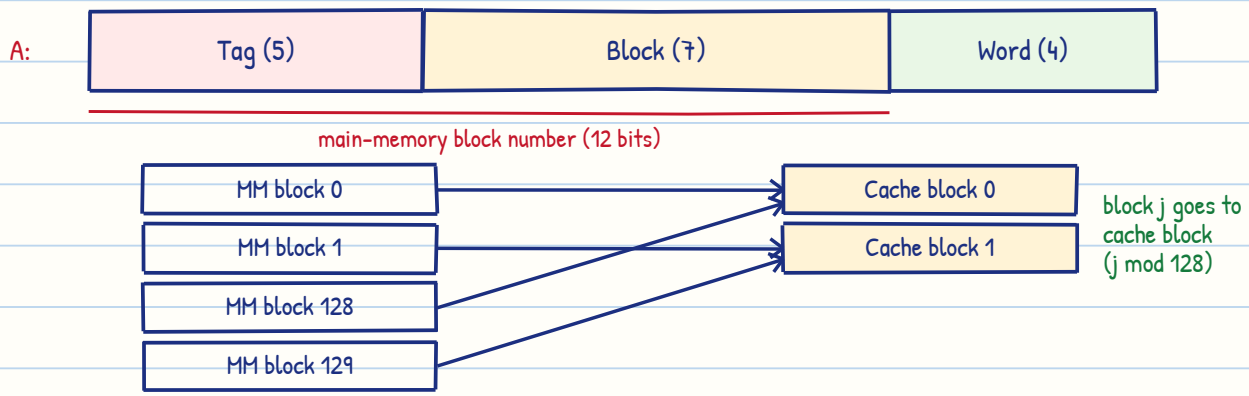


Fig 6: Direct-mapped cache

- **Hit check:** compare the tag in the address with the tag stored in that cache block (and check the **valid bit**).
- + Very simple and cheap. - Contention: two blocks that map to the same position keep replacing each other even if other cache blocks are empty.

Worked example (direct mapping)

Address 2FSC (hex) = 0010 1111 0101 1100. Word = last 4 bits = 1100 = 12. Block = next 7 bits = 1110101 = 117. Tag = first 5 bits = 00101 = 5. Check: memory block = 2FSC >> 4 = 757; 757 mod 128 = 117 and 757 / 128 = 5. So word is in cache block 117 if its stored tag = 5.

(b) Associative (fully associative) mapping

- A memory block can be placed in **any** cache block. Address = **Tag (12) | Word (4)** .

- The 12-bit tag is compared with **all 128 stored tags in parallel** . + Best use of cache, flexible. - Expensive hardware (128 comparators) and needs a replacement algorithm.

(c) Set-associative mapping

- Compromise: cache blocks are grouped into **sets** . A memory block maps to **one set** but may occupy **any block in that set** .
- Example **2-way** (2 blocks per set): 64 sets. Address = **Tag (6) | Set (6) | Word (4)** . Only the tags of the 2 blocks in the selected set are compared.
- 1-way set-associative = direct mapping; 128-way = fully associative. Real caches use 2-, 4- or 8-way.

Replacement algorithms

When the set/cache is full a block must be removed (needed only for associative and set-associative caches; direct mapping has no choice).

- **LRU** (Least Recently Used): remove the block unused for the longest time - works well due to locality; uses counters.
- **FIFO:** remove the oldest block. **Random:** remove a randomly chosen block (simple, fairly good). **LFU:** least frequently used.
- **Valid bit:** tells whether a cache block holds valid data (cleared at power-up). **Coherence:** if DMA changes main memory, the cached copy becomes stale and must be invalidated/flushed.

6. Performance Considerations

Performance depends on **hit rate (h)** and **miss penalty (M)** = extra time to bring a block from main memory.

Average access time

$t_{avg} = h \times C + (1 - h) \times M$ where C = cache access time, M = miss penalty. Example: h = 0.95, C = 1 cycle, M = 10 cycles -> $t_{avg} = 0.95 + 0.05 \times 10 = 1.45$ cycles (vs 10 cycles with no cache, i.e. about 7 times faster).

Two-level cache (L1 and L2)

$t_{avg} = h1\ C1 + (1 - h1)\ h2\ C2 + (1 - h1)(1 - h2)\ M$. Example: h1 = 0.95, h2 = 0.9, C1 = 1, C2 = 10, M = 100 -> 0.95 + 0.45 + 0.5 = 1.9 cycles.

Ways to improve performance

- **Higher hit rate:** larger cache, better mapping/replacement. Block size: bigger blocks help spatial locality but too big wastes space and raises miss penalty.
- **Interleaving:** main memory is split into several **modules** that work at the same time. With **low-order bits selecting the module** , consecutive words lie in different modules and a block can be fetched in parallel.

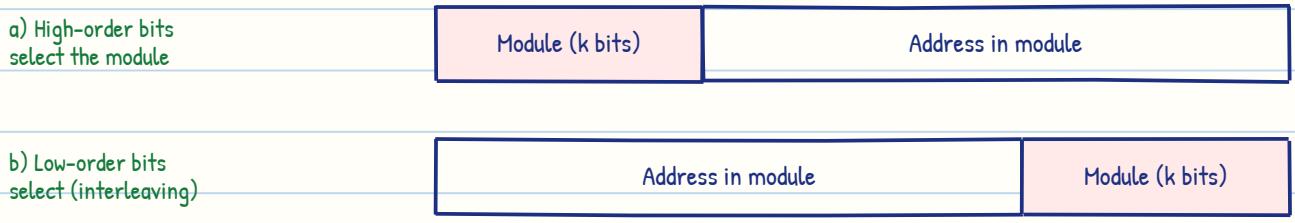


Fig 7: Address structure of memory modules

- **Example** (access = 8 cycles, transfer = 1 cycle, 4-word block): without interleaving 1 + 4 x (8 + 1) = **37 cycles** ; with 4-way interleaving 1 + 8 + 4 x 1 = **13 cycles** .
- **Write buffer:** processor puts writes in a small buffer and continues; the buffer writes to memory later.
- **Prefetching:** data is brought into the cache **before** it is requested (by special instructions or hardware).
- **Lockup-free (non-blocking) cache:** processor keeps working on other accesses while a miss is being served.
- **Separate L1 caches** for instructions and data, plus bigger **L2/L3** caches, are common.

7. Virtual Memories

Programs can be larger than main memory and many programs must share it. **Virtual memory** gives each program a large **virtual (logical) address space** ; only the needed parts are kept in main memory and the rest stays on disk. The OS and hardware move parts automatically.

- Addresses generated by the processor = **virtual addresses** . Addresses of real memory = **physical addresses** . The **MMU** (Memory Management Unit) translates virtual -> physical.
- Memory is divided into fixed-size **pages** (virtual) and **page frames** (physical), typically 2K-16K bytes.
- Virtual address = **virtual page number + offset** ; physical address = **page frame**

number + offset . The offset does not change.

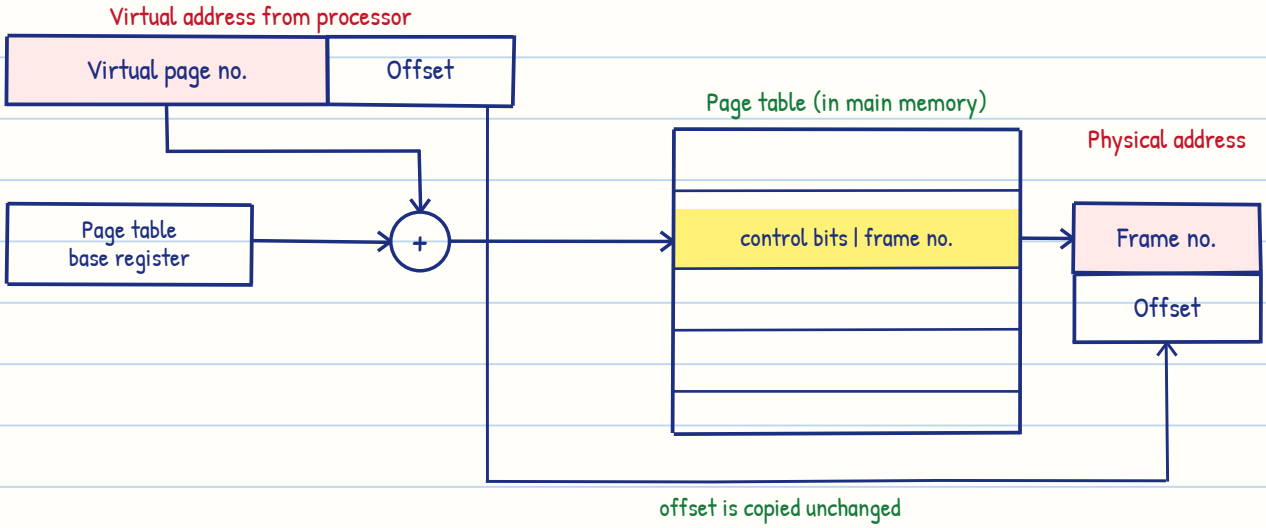


Fig 8: Virtual-to-physical address translation using the page table

Page table

- Stored in main memory. **Page table base register** + virtual page number = address of the page-table entry.
- Each entry has the **frame number** and **control bits** : **valid/present** bit, **modified (dirty)** bit, **reference** bit and **protection** (read/write) bits.

> **Example**
>
> 32-bit virtual address with 4 KB pages: offset = 12 bits, virtual page number = 20 bits, so the page table has 2^20 = 1M entries.

TLB - Translation Lookaside Buffer

- Reading the page table in memory for every access would **double** the access time. So the MMU keeps recently used entries in a small fast **associative cache** called the **TLB** .
- **TLB hit:** frame number is available at once. **TLB miss:** entry is read from the page table and put into the TLB.

Page fault and replacement

- If the valid bit = 0 the page is not in memory -> **page fault** (an exception). The **OS** loads the page from disk (**demand paging**); meanwhile the CPU runs another program.
- If memory is full a page is replaced (LRU-like). A **dirty** page is written back to disk first; clean pages are just overwritten. **Write-back** is always used because disk writes are very slow.

Cache	Virtual memory
Speeds up main memory access	Gives illusion of a large memory
Between CPU and main memory	Between main memory and disk
Managed by hardware	Managed by OS + MMU hardware
Block size ~ 16-128 bytes; miss costs ns	Page size ~ KBs; miss costs ms

8. Memory Management Requirements

Several programs run together (**multiprogramming**), so the OS must manage memory. The memory manager (OS + hardware) must provide:

1. **Relocation:** a program can be loaded anywhere in memory (it is not known in advance). Virtual addresses are translated to physical addresses at run time.
2. **Protection:** one program must not read or write another program's (or the OS's) memory. Done with **protection bits** in the page table and **base/limit** checks.
3. **Sharing:** programs may share code or data (e.g. a library or editor) - the same physical page is mapped into several virtual spaces.
4. **Logical organisation:** programs are written as modules (code, data, stack) - supported by **segmentation** .
5. **Physical organisation:** moving information between main memory and secondary storage automatically - supported by **virtual memory** .

Processor states for protection

- **User state:** normal programs run here; cannot execute **privileged instructions** (I/O, changing MMU, etc.).
- **Supervisor (system) state:** the OS runs here with full access. A user program enters it only through an **interrupt/system call** .

Paging	Segmentation
Fixed-size pages, invisible to programmer	Variable-size logical units (code, data, stack)
Causes internal fragmentation	Causes external fragmentation
Address = page no. + offset	Address = segment no. + offset
Page table	Segment table (base and limit)

9. Secondary Storage

Secondary storage is **non-volatile, large and cheap but slow** . It holds programs and data permanently (hard disk, optical disk, tape, flash).

9.1 Magnetic hard disk

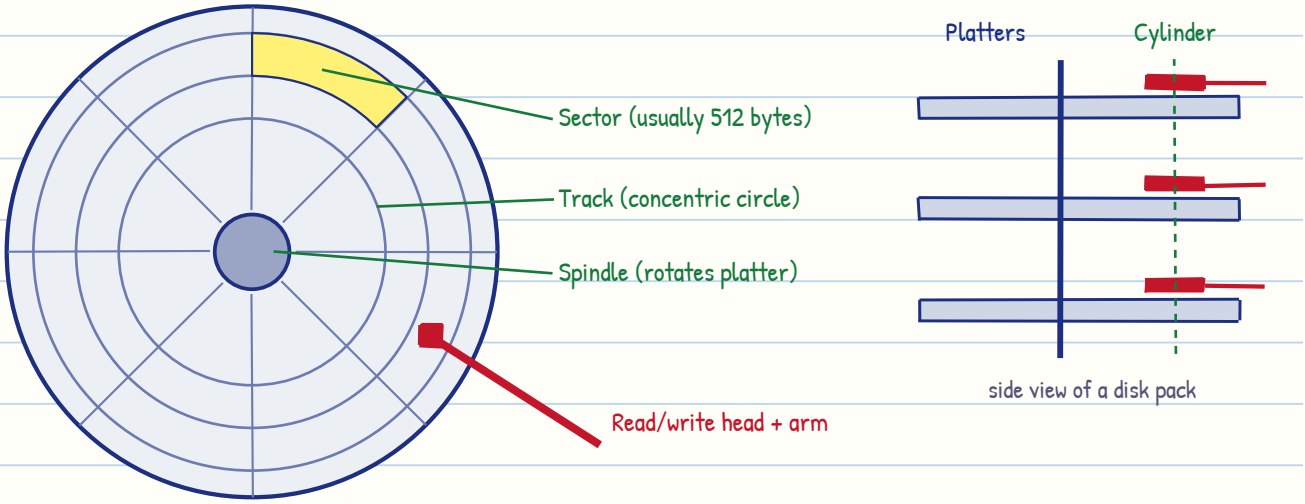


Fig 9: Disk organisation

- A disk pack has several **platters** on a **spindle** rotating at constant speed (5400 / 7200 / 15000 rpm). Each surface has a **read/write head** .
- Data is stored in **tracks** (concentric circles); each track is divided into **sectors** (usually 512 bytes). The same track on all surfaces = **cylinder** .
- A sector is addressed by **surface (head) no., track (cylinder) no. and sector no.**

Disk access time = Seek time + Rotational delay + Transfer time

Seek time: moving the head to the correct track (about 5-10 ms). **Rotational delay (latency):** waiting for the sector to arrive under the head = half a rotation on average.

Example: 7200 rpm = 120 rev/s -> one rotation = 8.33 ms -> average latency = 4.17 ms. With 6 ms seek, access is about 10 ms + a tiny transfer time.

- **Disk controller** sits between the disk and the bus: it controls seeking, reads/writes sectors, does **error checking (ECC)** and transfers data using DMA.

9.2 RAID (Redundant Array of Independent Disks)

Several disks are used together for **speed and reliability** .

- **RAID 0:** data **striped** across disks -> fast, but **no redundancy** .
- **RAID 1: mirroring** - every disk has a copy -> reliable but doubles the cost.
- **RAID 5:** striping with **distributed parity** -> survives failure of any one disk; good

balance of speed, safety and cost.

9.3 Optical, tape and flash

Device	Key points
CD-ROM / CD-R / CD-RW	Laser reads pits and lands; ~650-700 MB; read-only / write-once / rewritable
DVD	Shorter-wavelength laser, 4.7 GB per layer (more with two layers/sides)
Blu-ray	Blue-violet laser, about 25 GB per layer
Magnetic tape	Sequential access only; very cheap; used for backup and archives
Flash / SSD	No moving parts, fast random access, shock-proof; costlier per GB than disk

Quick Revision and Important Questions

- Memory hierarchy: registers > cache > main memory > disk (size up, speed down, cost/bit down).
- SRAM = latch, no refresh, fast. DRAM = capacitor, refresh, cheap. SDRAM/DDR = clocked, burst, double rate.
- ROM family: ROM, PROM, EPROM, EEPROM, Flash. Mapping: direct, associative, set-associative.
- $t_{avg} = hC + (1 - h)M$. Virtual address = page no. + offset; TLB caches page-table entries.

Likely exam questions

1. Explain the organisation of a memory chip with a neat diagram.
2. Differentiate SRAM and DRAM. Explain the working of a DRAM cell and refresh.
3. Explain the types of ROM.
4. Explain the memory hierarchy and the concept of locality of reference.
5. Explain direct, associative and set-associative mapping with address formats.
6. Explain write-through and write-back; replacement algorithms.
7. How does interleaving improve memory performance? Derive average access time.
8. Explain virtual memory, address translation, page table and TLB.
9. Explain the memory management requirements.
10. Explain disk organisation and access time; write short notes on RAID and optical disks.

UNIT - V

INPUT / OUTPUT ORGANIZATION

Computer Organization - Handwritten Study Notes

Syllabus topics:

Accessing I/O Devices, Interrupts, Processor Examples, Direct Memory Access, Buses, Interface Circuits, Standard I/O Interfaces

1. Accessing I/O Devices

Input/output (I/O) devices - keyboard, mouse, display, printer, disk, network card - let the computer talk to the outside world. They are connected to the processor and memory through a **bus** .

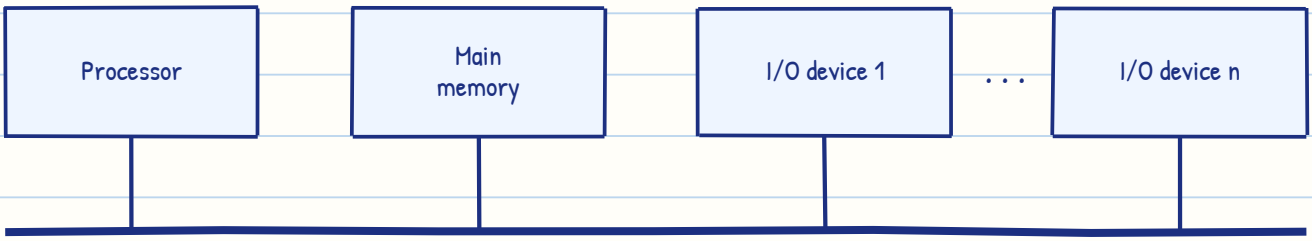


Fig 1: A single-bus structure

- The bus has **address lines, data lines and control lines** . Every I/O device is given an **address** (just like a memory location), so the processor can select it.

Two ways of addressing I/O devices

- **Memory-mapped I/O:** I/O devices and memory share **one address space** . Any instruction that accesses memory (Move, Load, Store) can also read or write a device register. Example: Move DATAIN, R0 reads the keyboard; Move R0, DATAOUT writes to the display. No special instructions are needed.
- **I/O-mapped (isolated) I/O:** I/O devices have a **separate address space** and the processor has special **IN / OUT** instructions. A control line (**IO/M**) tells whether the address is for I/O or memory.

Memory-mapped I/O	Isolated (I/O-mapped) I/O
One address space for memory + I/O	Separate address spaces
Normal Load/Store/Move instructions	Special IN / OUT instructions
Uses up some memory addresses	Full memory address space is free
Used by ARM, most RISC processors	Used by Intel x86 (and also memory-mapped)

I/O interface

The **I/O interface** is a circuit between the bus and the device. The device is much slower and works differently from the processor, so the interface adapts them.

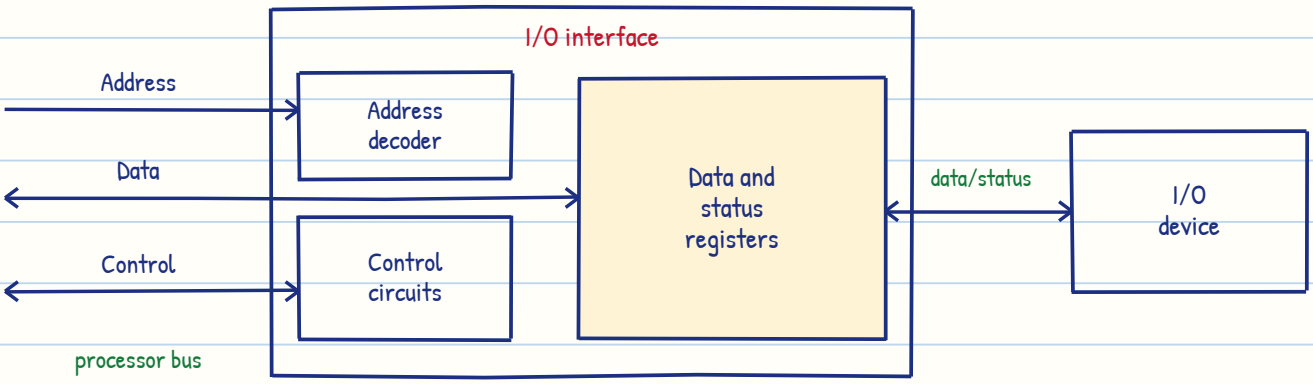


Fig 2: I/O interface for an input/output device

- **Address decoder:** recognises the device address on the address lines.
- **Data register** (e.g. **DATAIN** , **DATAOUT**) holds the data being transferred. **Status register** holds flags such as **SIN** (input data ready) and **SOUT** (output device ready).
- **Control circuits:** handle the timing and control signals.

Program-controlled I/O (polling)

The processor **repeatedly checks (polls)** the status flag until the device is ready, then transfers the data. Example: read one character from the keyboard and echo it on the display.

```
READWAIT: Branch to READWAIT if SIN = 0
          Move DATAIN, R1 ; read the character
WRITEWAIT: Branch to WRITEWAIT if SOUT = 0
          Move R1, DATAOUT ; display the character
```

- **Drawback:** the processor wastes all its time in the waiting loop (busy waiting). Devices are thousands of times slower than the CPU.
- **Solutions:** use **interrupts** (device calls the processor when ready) or **DMA** (block transfer without the processor).

2. Interrupts

Instead of waiting, the processor does useful work. When the device is ready it sends an **interrupt-request** signal on a control line. The processor stops the current program temporarily, runs a special routine, and then continues.

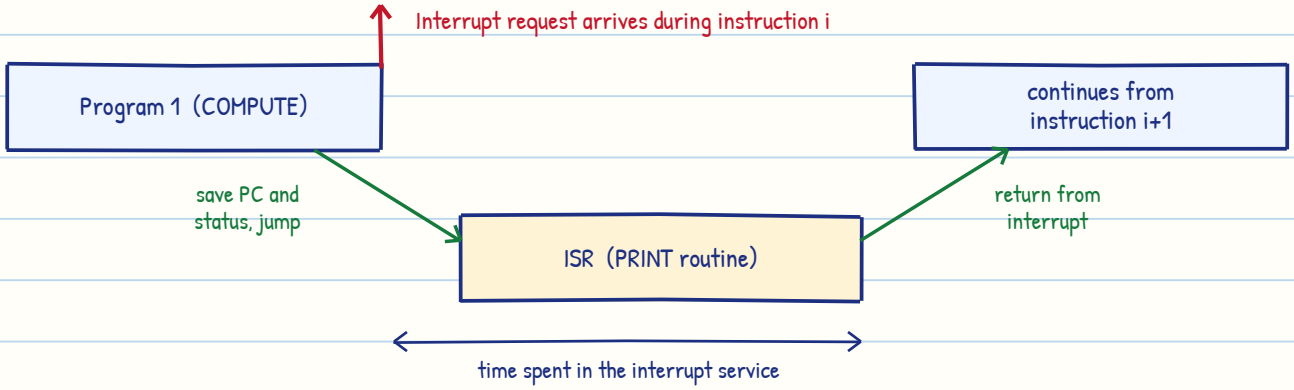


Fig 3: Transfer of control through an interrupt

What happens on an interrupt

1. Processor **completes the current instruction** (instruction i).
2. It **saves the return address (PC)** and the processor status (PS), then **disables further interrupts** (IE = 0).
3. It jumps to the **Interrupt Service Routine (ISR)** for that device. The ISR serves the device and **removes the request** .
4. A **Return-from-interrupt** instruction restores PC and PS and the program continues at instruction i+1.

- **Interrupt latency** = delay between the arrival of the request and the start of the ISR.
- ISR vs subroutine: a subroutine is **called by the program** at a known place; an ISR can run at **any time** (unplanned), so it must save and restore every register it uses, and the interrupted program never knows.

Enabling and disabling interrupts

- **IE bit** in the processor status register: if 0, interrupt requests are ignored (e.g. while an ISR runs or during a critical section).
- Each device interface has its own **interrupt-enable bit** in its control register (e.g. **KEN** for keyboard, **DEN** for display), so a device can be allowed or blocked.
- **Interrupt hardware:** all devices share one request line **INTR** (wired-OR). The processor answers with **INTA** (interrupt acknowledge).

Handling many devices

When several devices share the INTR line the processor must find out **who interrupted** and **who goes first** .

1. **Polling the status registers:** the ISR checks the IRQ bit of every device in turn. Simple, but wastes time.
2. **Vectored interrupts:** the device sends a **vector** (its code, or the starting address of its ISR) on the data lines when INTA arrives. The processor jumps directly to the right ISR - **fast** .
3. **Interrupt nesting:** a high-priority device may interrupt the ISR of a low-priority device. The processor has a **priority level** in PS and accepts a request only if its priority is **higher** than the current one.

Priority and simultaneous requests

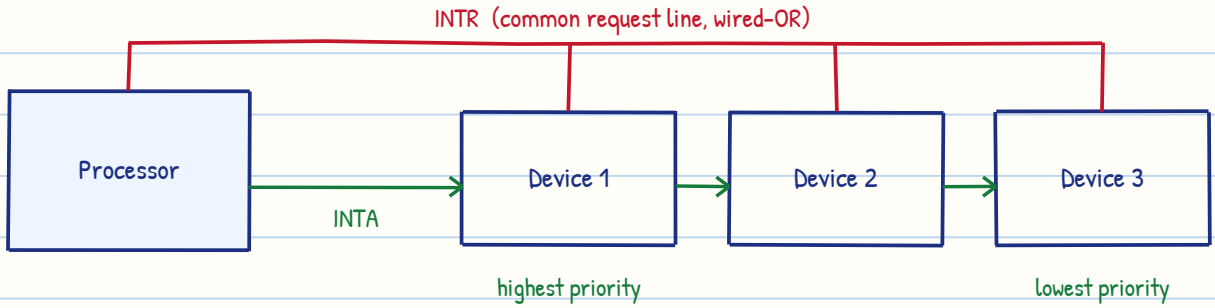


Fig 4: Daisy-chain interrupt priority

- **Daisy chain:** INTA is passed from device to device. A device that has requested keeps INTA and puts its vector; otherwise it passes INTA on. The device **nearest the processor has the highest priority** .
- **Separate INTR/INTA lines** for each device (independent priority) or **groups of daisy chains** are also used, giving flexible priorities.

Exceptions

Exceptions are interrupts generated inside the processor (not by I/O devices) and are handled by similar ISRs.

- **Recovery from errors:** illegal instruction, arithmetic overflow, divide by zero, memory (ECC) error, page fault.
- **Debugging: trace mode** (exception after each instruction) and **breakpoints** (special instruction) let a debugger take control.
- **Privilege exception:** a user program tries to run a privileged instruction (I/O, change of interrupt mask) - only the OS may do it.

3. Processor Examples

The same ideas (device registers, status flags, interrupt vectors) are used by real processors. Two common examples are **ARM** and **Intel IA-32 (Pentium)** .

3.1 ARM

- **I/O is memory-mapped:** device registers are accessed with normal Load/Store instructions at fixed addresses. Parallel-port and serial-port interfaces use data, status and control registers.
- ARM has two interrupt levels: **IRQ** (normal) and **FIQ** (fast interrupt, higher priority).
- **CPSR** (current program status register) has the **I bit** (1 = IRQ disabled), the **F bit** (1 = FIQ disabled) and the **mode bits** .
- **Banked registers:** each interrupt mode has its own copies of some registers. IRQ mode has its own R13 (SP), R14 (LR) and SPSR; **FIQ mode also has its own R8-R12** , so the FIQ routine need not save registers -> very fast.

ARM interrupt response

1. Finish the current instruction; copy **CPSR into SPSR** of the new mode.
2. Change to **IRQ/FIQ mode** , set the **I bit** (and F bit for FIQ) to disable further interrupts.
3. Store the **return address in R14 (LR)** of that mode.
4. Load **PC with the vector address** . Return: SUBS PC, LR, #4 (also restores CPSR from SPSR).

Exception	Vector address
Reset	0x00
Undefined instruction	0x04
Software interrupt (SWI)	0x08
Prefetch abort	0x0C
Data abort	0x10
IRQ	0x18
FIQ	0x1C

- A **vectored interrupt controller** is added when many devices share IRQ.

3.2 Intel IA-32 (Pentium)

- Supports **both** memory-mapped I/O and **isolated I/O** . Isolated I/O uses **IN** and **OUT** instructions with a **16-bit port address** (up to 64K ports); the port address is an

immediate value or in register **DX** .

- **Interrupt pins: INTR** (maskable, controlled by the **IF** flag; **STI** sets IF, **CLI** clears IF) and **NMI** (non-maskable).
- **256 interrupt types** (vectors 0-255): hardware interrupts, software interrupts (**INT n**) and exceptions (0 = divide error, 2 = NMI, 3 = breakpoint ...).
- The type number indexes the **Interrupt Descriptor Table (IDT)** (4-byte entries in real mode, 8-byte descriptors in protected mode) that gives the ISR address.
- On an interrupt the processor **pushes EFLAGS, CS and EIP** on the stack and clears IF. **IRET** pops them and returns.
- A **programmable interrupt controller (8259A)** handles up to 8 devices (cascade for 15). It sends the vector number to the CPU during **INTA** and handles priorities.

Feature	ARM	Intel IA-32
I/O method	Memory-mapped only	Isolated (IN/OUT) and memory-mapped
Interrupt inputs	IRQ and FIQ	INTR and NMI
Disable flag	I and F bits in CPSR	IF flag in EFLAGS
Vector table	Fixed addresses (0x00-0x1C)	256 entries (IDT)
Return	SUBS PC, LR, #4	IRET

4. Direct Memory Access (DMA)

In program-controlled and interrupt-driven I/O the processor moves **every word** , which is too slow for large blocks (disk, network). **DMA** lets a special **DMA controller** transfer a whole block **directly between the I/O device and main memory without the processor** .

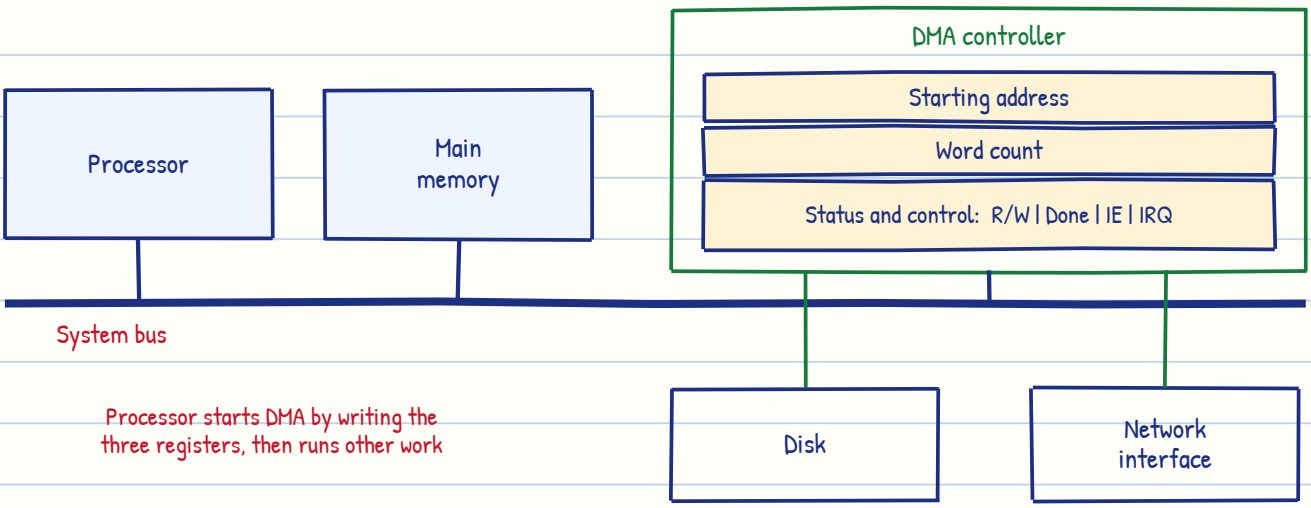


Fig 5: DMA controller in a computer system

DMA controller registers

- **Starting address register** - memory address where the block starts.
- **Word count register** - number of words still to transfer.
- **Status and control register** - contains **R/W bit** (direction of transfer), **Done flag** (set when the block is complete), **IE** bit (interrupt enable) and **IRQ** bit (interrupt request).

Steps of a DMA transfer

1. Processor (through the OS) **loads the three registers** and starts the DMA transfer.
2. The processor continues with **another program** ; the program that wanted I/O waits.
3. DMA controller transfers words between device and memory, **incrementing the address and decrementing the word count** for each word.
4. When the count reaches 0, it sets **Done** and, if **IE = 1** , sends an **interrupt** to the processor. The OS then resumes the waiting program.

Memory access sharing

- Processor and DMA both need the memory bus. The **DMA controller gets priority** because the device must not lose data.
- **Cycle stealing:** DMA takes **one memory cycle at a time** and gives the bus back - processor slows slightly.
- **Block (burst) mode:** DMA keeps the bus for the **whole block** - fast for the device, but the processor is blocked meanwhile.
- **Transparent mode:** DMA uses the bus only when the processor is not using it - no slowdown, but more hardware.

Note: Because DMA writes memory directly, a copy of that data in the cache may become stale (cache coherence). The OS must invalidate or flush the cache for DMA buffers.

Bus arbitration

Only **one device (bus master)** can control the bus at a time. **Arbitration** decides who becomes master. The processor is normally the master; a DMA controller must **request** the bus.

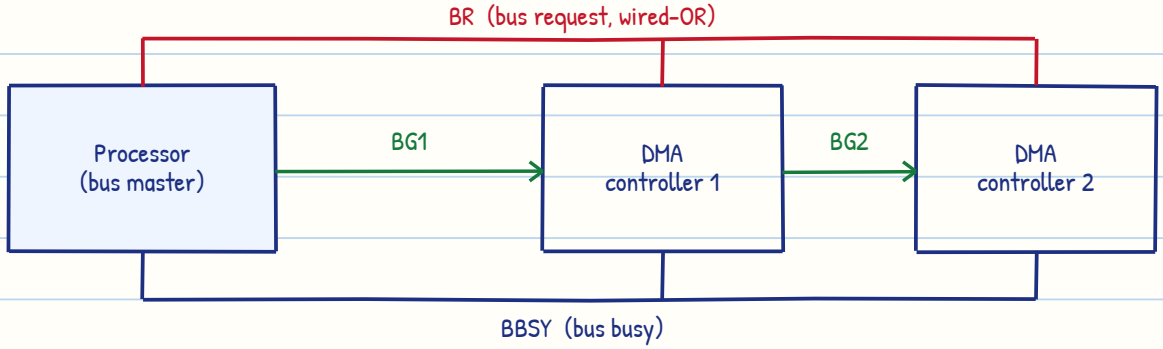


Fig 6: Centralised daisy-chain arbitration

- **BR** (Bus Request) is raised by a device. The processor finishes its bus cycle and sends **BG** (Bus Grant), which passes along the daisy chain. The device that requested keeps BG, raises **BBSY** (busy) and becomes master. It drops BBSY when finished.
- **Centralised arbitration:** one **arbiter** (usually in the processor) grants the bus - by **daisy chain** or **independent BR/BG pair per device** (faster, more wires).
- **Distributed arbitration:** no central arbiter. Each device has a **4-bit ID** and puts it on shared **open-collector lines** (wired-OR). Each device compares the bus value with its ID; a device that sees a higher ID stops driving its lower bits. Finally the bus shows the **highest ID, which wins** . Example: IDs 0101 (5) and 0110 (6) -> bus = 0110, device 6 wins.

	Polling	Interrupt	DMA
Who moves data	Processor	Processor	DMA controller
Processor busy?	Fully (waiting)	Only during ISR	Only at start and end
Best for	Simple, fast devices	Slow, occasional devices	Large blocks (disk, network)

5. Buses

A **bus** is a set of wires that carries **address, data and control** signals between the processor, memory and I/O devices. The **bus protocol** is the set of rules for timing and signals so that the **master** (initiator) and **slave** (target) understand each other.

- **Bus master** starts a transfer (processor or DMA); the addressed **slave** responds.
- **Bandwidth** = bus width x clock rate. Example: 64-bit bus at 100 MHz = 8 bytes x 100 M = **800 MB/s** .
- Two timing methods: **synchronous** (common clock) and **asynchronous** (handshake).

5.1 Synchronous bus

All devices use a **common bus clock** . Every action happens at a fixed clock edge.

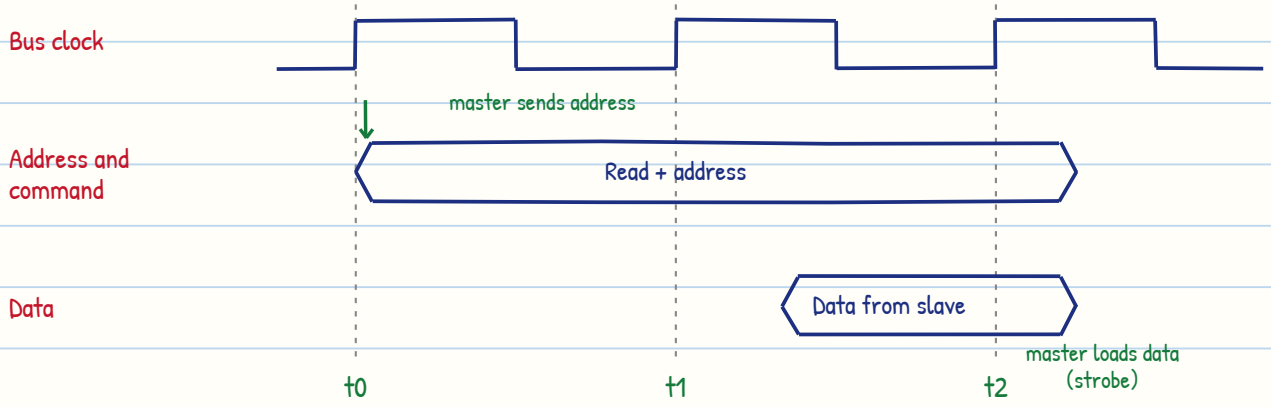


Fig 7: Timing of an input (read) transfer on a synchronous bus

- **t0:** master puts the **address and the Read command** on the bus.
- **t1:** the addressed slave decodes the address and **places the data** on the data lines (after its access time).
- **t2:** master **loads (strobes) the data** into its register and then removes address and command.
- The clock period must be longer than the **maximum propagation delay + set-up time** , so the clock is set for the **slowest** device.
- **Multiple-cycle transfer:** if the slave is slow, it keeps a **Slave-ready** signal low and the master waits for extra clock cycles - so slow and fast devices can share one bus.

5.2 Asynchronous bus

No common clock. The master and slave use two lines, **Master-ready** and **Slave-ready** , in a **full handshake** .

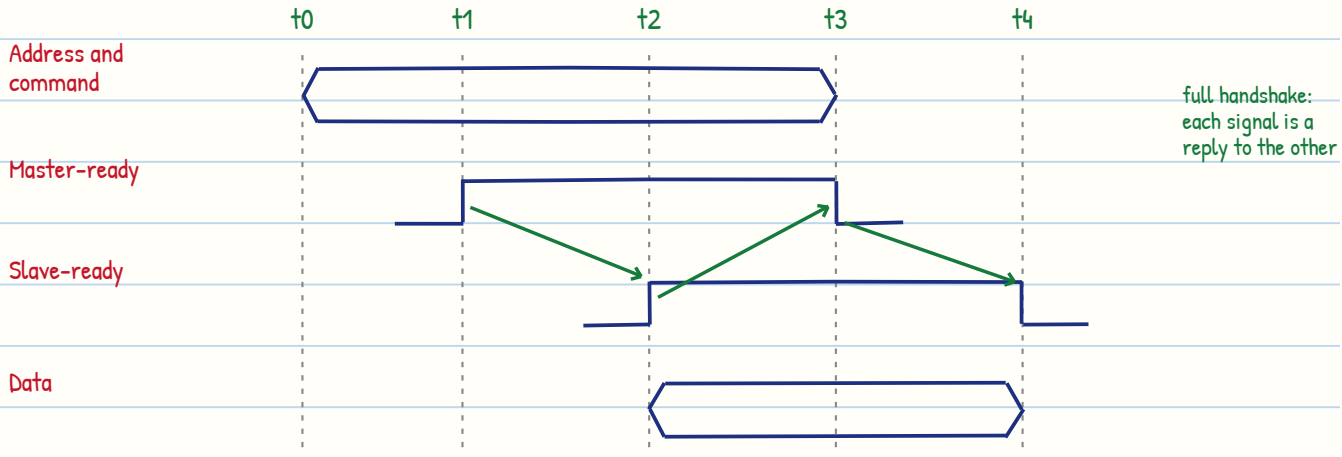


Fig 8: Handshake for an input (read) transfer

1. **t0:** master puts address and command on the bus.
2. **t1:** after a small delay (skew), master asserts **Master-ready** - "address is valid".
3. **t2:** the selected slave performs the operation, puts the data on the bus and asserts **Slave-ready** .
4. **t3:** master loads the data, removes address/command and drops Master-ready.
5. **t4:** slave removes the data and drops Slave-ready. The bus is free.

Synchronous bus	Asynchronous bus
Common clock controls timing	Handshake signals control timing
Simple, fast interface logic	More complex circuits
Speed fixed by the slowest device	Speed adapts to each device
Poor for devices of very different speed	Handles devices of widely different speed
Clock skew limits bus length	Handshake delay slows each transfer

- Large systems use a **bus hierarchy** : a fast processor bus and memory bus, joined by **bridges** to slower I/O buses. Wide buses and fast clocks give higher bandwidth.

6. Interface Circuits

An interface circuit connects an I/O device to the bus. It has **data registers, status flags, an address decoder** and control logic. Interfaces are of two kinds: **parallel port** and **serial port** .

6.1 Parallel interface

All bits of a data item (e.g. 8 bits) are transferred **at once** on separate lines. Fast, but needs many wires - good for short distances (printer, keyboard).

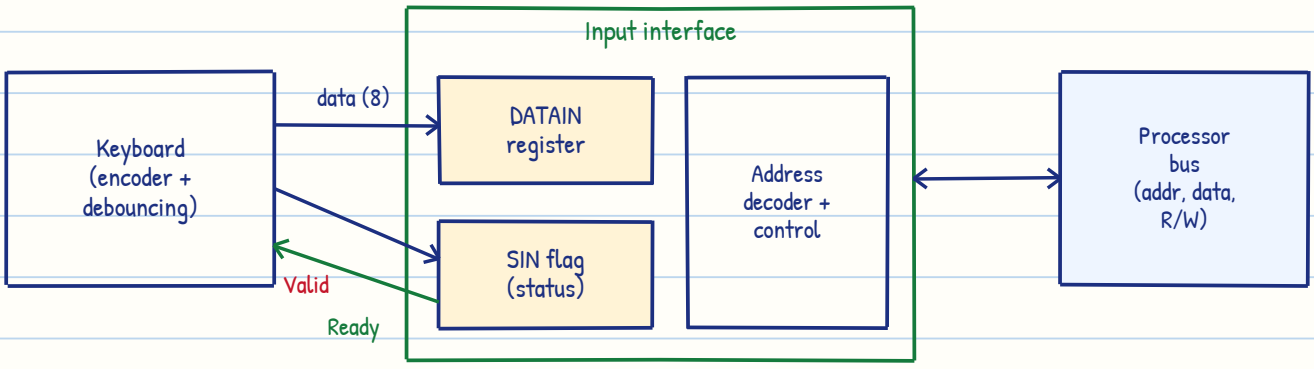


Fig 9: Input interface for a keyboard

- **Keyboard (input):** when a key is pressed the encoder produces the code and a **Valid** signal. The interface loads the code in **DATAIN** and sets **SIN = 1** . The processor reads DATAIN (which clears SIN) and the interface sends **Ready** back, telling the keyboard to

send the next code.

- **Printer (output):** the processor writes **DATAOUT** ; **SOUT** is cleared and the interface sends **Valid** to the printer. When the printer is **idle** again it replies and the interface sets **SOUT = 1** .
- **General parallel port:** each of its 8 lines can be input or output according to a **Data Direction Register (DDR)** , and two **handshake control lines (C1, C2)** can be set up for different modes and can raise interrupts.

6.2 Serial interface

Data is sent **one bit at a time** over a single line. It needs fewer wires, so it suits **long distances** (modem, mouse, network, USB).

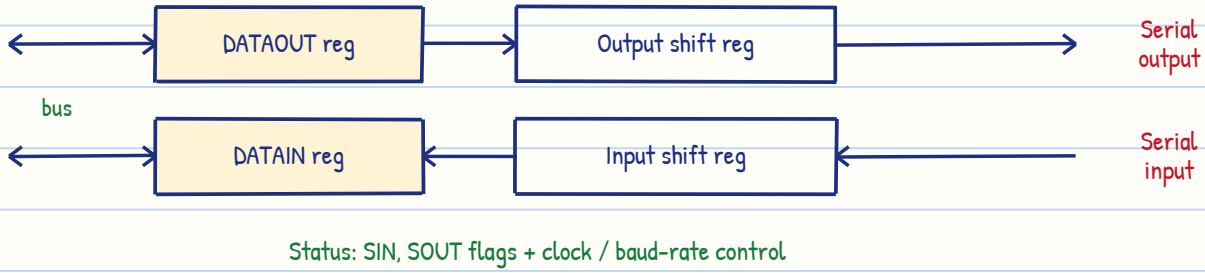


Fig 10: Serial interface and an asynchronous character

- Output: the processor writes a word into **DATAOUT** ; it is copied to the **output shift register** , which shifts the bits out one by one. Input: bits shift into the **input shift register** , then the whole word moves to **DATAIN** and **SIN** is set.
- **Asynchronous serial:** each character has a **start bit (0)** , 5-8 **data bits** , optional **parity bit** and **1 or 2 stop bits (1)** . The idle line is 1. The receiver detects the start bit and samples every bit in the **middle** of its time using a clock **16 times faster** than the bit rate.
- **Baud rate** = signalling speed. Example: 9600 baud with 8N1 (10 bits/char) -> **960 characters/s** . The chip that does this is a **UART** .
- **Synchronous serial:** no start/stop bit for each character; a block of characters is sent with **synchronisation characters** and the receiver recovers the clock - more efficient for big blocks.

Parallel	Serial
Many bits at a time	One bit at a time
Many wires, short distance	Few wires, long distance
Fast for short links; cross-talk problems	Very high speed possible with good signalling
e.g. printer port, internal buses	e.g. USB, Ethernet, SATA, PCI Express

7. Standard I/O Interfaces

A **standard** lets devices from different manufacturers work with any computer that follows it. I/O devices can be connected in three ways: on the **processor bus** (fast, but processor-specific), on a **backplane bus** (standard, processor independent - **PCI**), or via an **I/O bus** (**SCSI, USB**).

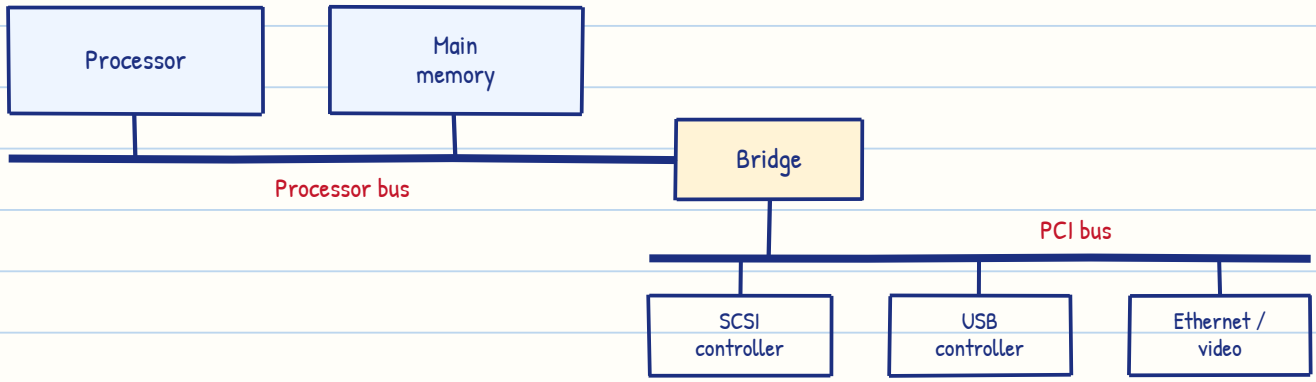


Fig 11: Bus hierarchy with a PCI bridge

7.1 PCI (Peripheral Component Interconnect)

- Developed by **Intel (about 1992)** ; processor independent. A **bridge** connects the processor bus to the PCI bus, so changing the processor does not change the PCI devices.
- Width **32 bits** (extendable to 64) at **33 MHz** (or 66 MHz): 32 x 33 MHz = **132 MB/s** ; 64-bit at 66 MHz = **528 MB/s** .
- **Plug and play:** every device has a small **configuration space/ROM** holding its type, memory and interrupt needs; the system configures it automatically at start-up.
- Address and data lines are **multiplexed** to save pins; supports **burst transfers** ; has a **central arbiter** . Devices act as **initiators or targets** .
- **PCI Express** is the later serial, point-to-point version using **lanes** .

7.2 SCSI (Small Computer System Interface)

- An **ANSI standard** bus for disks, tapes, CD drives and scanners.
- **Narrow SCSI** is 8 bits wide (up to **8 devices** , IDs 0-7 including the controller); **wide**

SCSI is 16 bits (up to **16 devices**).

- Devices are connected in a **daisy chain** (one cable, **terminators** at both ends). Each device has a **unique ID** ; a device acts as **initiator** (controller) or **target** (disk).
- A transfer goes through **phases** : arbitration -> selection -> information transfer (command, data, status, message) -> bus free. Both asynchronous and synchronous modes exist. Fast versions reach **320 MB/s** (Ultra320).

7.3 USB (Universal Serial Bus)

- Designed by a group of companies (Intel, Microsoft, HP, NEC, Philips and others) as a **simple, low-cost, plug-and-play, hot-pluggable** way to connect keyboards, mice, printers, cameras, drives etc.

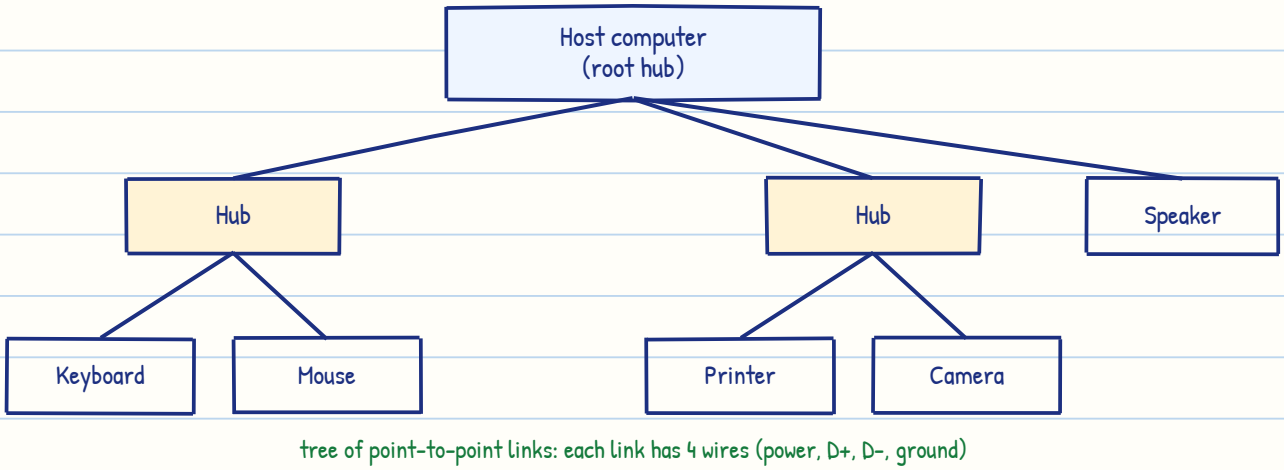


Fig 12: USB tree structure with hubs

- **Tree of hubs:** the **root hub** is in the host computer; hubs have more ports. Up to **127 devices** can be attached. Each device gets a **7-bit address** when plugged in (**enumeration**).
- Cable: **4 wires - +5 V power, D+, D- (differential data pair), ground** . The bus can **power small devices** .
- Speeds: **Low 1.5 Mbps** (mouse, keyboard), **Full 12 Mbps** , **High 480 Mbps** (USB 2.0), **SuperSpeed 5 Gbps** (USB 3.0).
- **Communication:** the host **polls** ; all transfers use **packets - token** (IN, OUT, SETUP), **data** and **handshake** (ACK, NAK, STALL). Data is coded with **NRZI + bit stuffing** for clock recovery. Time is split into **1 ms frames** .
- **Transfer types: Control** (setup), **Bulk** (printers, disks), **Interrupt** (keyboard/mouse - polled regularly), **Isochronous** (audio/video - fixed rate, no retry).
- **Plug and play:** when a device is connected, the hub tells the host; the host reads its descriptor, assigns an address and loads the right driver.

	PCI	SCSI	USB
Type	Backplane (internal) bus	I/O bus, daisy chain	Serial I/O bus, tree
Data path	32/64-bit parallel	8/16-bit parallel	1 bit (differential pair)
Devices	Add-in cards	8 or 16 (disks, tape)	Up to 127 (many kinds)
Plug and play	Yes (configuration)	Limited	Yes, hot-pluggable

Note: Other standards you may meet: FireWire (IEEE 1394), SATA (serial disk interface), Thunderbolt, PCI Express.

Quick Revision and Important Questions

- Access: memory-mapped (normal instructions) vs isolated (IN/OUT). Polling wastes CPU time -> use interrupts / DMA.
- Interrupt: finish instruction -> save PC and PS -> ISR -> return. Vectored = fast; daisy chain = priority; exceptions = internal.
- DMA registers: address, word count, status/control. Modes: cycle stealing, burst, transparent. Arbitration: centralised or distributed.
- Synchronous bus = clock; asynchronous = Master-ready / Slave-ready handshake. Serial frame = start + data + stop.
- PCI (backplane, plug and play), SCSI (daisy chain, 8/16 devices), USB (tree, hubs, 127 devices).

Likely exam questions

1. Explain memory-mapped and isolated I/O. Describe the I/O interface with a diagram.
2. Explain program-controlled I/O with an example program.
3. Explain interrupt handling. Discuss vectored interrupts, nesting and priority (daisy chain).
4. What are exceptions? Give examples.
5. Explain interrupt handling in ARM and Intel processors.
6. Explain DMA with a block diagram. Describe the DMA registers and transfer modes.
7. Explain bus arbitration (centralised and distributed).
8. Compare synchronous and asynchronous buses with timing diagrams.
9. Explain parallel and serial interface circuits.
10. Write short notes on PCI, SCSI and USB.